MettaKernel and MeTTa Native: A Logic-Polymorphic Native-Calculus Architecture

Dependent Types, Proof-Relevant Interlingua, and Admitted Execution

Oruži Zar Goertzel

2026-06-18 (design draft); realized-architecture update 2026-06-23;
Metamath-profile update 2026-08-06; NIK/native-theory updates 2026-08-14–23

Abstract

We set out to build one small *trusted* proof/inference kernel, written *in MeTTa* (as an ordinary MeTTa program, executed on the CeTTa engine), that can express and check proofs in *both* foundations we care about: CIC-grade *dependent type theory* (the foundation of Lean and Coq) and *higher-order Tarski–Grothendieck set theory* (HOTG, the classical higher-order logic of Megalodon). The implemented $\lambda\Pi$ checker is one important realization, but the theory has since become more general: the Native Inference Kernel (NIK) is a logic-polymorphic family of authorities and realizations, while the MeTTa Inference Kernel (MIK) is its MeTTa-hosted realization. A separately derived MeTTa-native type theory is one guest and one possible bootstrap language, not the definition of NIK. The kernel is a MeTTa program — not a new CeTTa language substrate — in the same spirit as `pverify`, an existing Metamath proof checker written in MeTTa. Three principles govern the work. First, **prototype a fully working kernel before any deeper integration into the engine**, so that the design is chosen empirically rather than frozen in advance. Second, **conversion is the danger line**: the kernel's $\beta\eta\delta\iota$ conversion must be its own small, decidable, resource-bounded MeTTa functions and must never be delegated to CeTTa's general runtime rewriting, or proof checking ceases to be a terminating judgment. Third, **use the native calculus licensed by each guest's proved structure**: compute judgments directly when decidable, compose admitted sound operations without interior rechecking, preserve native proof objects when they are genuine mathematical data, and request certificates only at explicit search or trust boundaries. We present the two reference kernels we mirror (the Lean `PureKernel` for DTT and Megalodon's `tp/tm/pf+check_doc` core for HOTG), a graded *curriculum* of the easiest examples that pin down the smallest fully expressive kernel (a DTT ladder following Smolka & Brown's *Introduction to Computational Logic*, and an HOTG ladder following Megalodon), and a prototype-first build plan whose first milestone is an honest *capacity ledger* (Agree/Diverge/Open) rather than a claim of finality. The MeTTa kernel's verdicts are treated as a *subject* to be measured against the reference authorities, not as an oracle itself. We also state the proved relationship among evolving Zero/Prime/HE/PeTTa dialect points, intrinsic `PureKernel`, the locally nameless Pattern Pure presentation, proof-relevant NIK, and the strictly descending bootstrap tower. The account distinguishes closed formal interfaces from executable fragments and from remaining adequacy obligations.

1 Why a single kernel for two foundations

Lean/Coq rest on the Calculus of Inductive Constructions: dependent function and pair types, an inductive-definition mechanism with computation rules, a universe hierarchy, and a *decidable* definitional equality. Megalodon rests on HOTG: simple types with an impredicative `Prop`, proof terms, classical logic, set membership, extensionality, foundational ($\in$-)induction, and a choice operator.

These are different mathematics, but a *proof kernel* for either is small: it is a term/type/proof object language plus a conversion relation plus a checking judgment, with everything else (parsing, automation, optimization, the ambient runtime) living *outside* the trusted core.

Our thesis — developed at length in the companion memo *metta_proof_kernel_grounded* — is that MeTTa is an apt host for such a kernel *as a program*: terms, types, and proofs are MeTTa atoms; the checking and conversion judgments are MeTTa functions; theories and imports are MeTTa data. The trusted waist is then a small auditable MeTTa module, and the two foundations appear as two *checkers* sharing that waist. We are not proposing to encode both foundations into one ambient rewriting theory (the Dedukti route); we keep the foundations' judgments first-class and only share the machinery beneath them.

What this document is. A design + curriculum memo to drive a prototype-first build. It records: the reference kernels we mirror (§2); the MeTTa-program architecture (§3); the graded curriculum that defines "smallest fully expressive" (§4); the build plan and its first milestone, a capacity ledger (§5); the conversion discipline that is the central hazard (§8); the open questions we deliberately leave open (§9); and — added in this revision — the *realized architecture and results* (§6) recording the as-built kernel that now exists, including where realization refined the plan; and the generalized proof-aware architecture (§7), which records what is now formalized about NIK, the MeTTa-native type theory, dialect regions, and their exact boundaries.

2 The two reference kernels (calibration authorities and templates)

2.1 DTT: the Lean `PureKernel`

A trusted, *sorry-free* dependently-typed kernel already exists in Lean (`Mettapedia/.../MeTTa/PureKernel/`, 28 files). Its term language is

```
PureTm n ::= var (Fin n) | const DeclName | U0 | U1
           | pi A B | sigma A B | id A t u
           | lam B | app f a | pair t u | fst p | snd p | refl t
```

with two universes (`U0 : U1`), Π, Σ, and identity types. It provides one-step $\beta/\delta/\iota$ reduction, Church–Rosser confluence, `HasType`, a *decidable* definitional equality (`defEqByNormalization?`), subject reduction, and a bidirectional checker. Ordinary families (`Bool`, `Nat`, `Unit`) are *not* hard-coded: they enter through a general declaration mechanism (`DeclSpec`, `InductiveDecl`, `RecursorDecl`) with full well-formedness proofs. This is both our DTT calibration authority and our design template; crucially, the existing runtime-to-kernel bridge marks `directExecEquivalence` as `false`, so the runtime is *not* yet known to agree with this kernel.

2.2 HOTG: the Megalodon `mathdata` core

Megalodon's trusted core (`repos/megalodon-1.13/src/mathdata.mli`) is genuinely compact — three small inductive types and a checker:

```
tp ::= TpVar i | Prop | Base i | Ar(tp,tp) | TpAll tp        (* simple types + type-vars
    + type-forall *)
tm ::= DB i | TmH h | Prim i | Ap(tm,tm) | Lam(tp,tm)
     | Imp(tm,tm) | All(tp,tm) | TTpAp(tm,tp) | TTpLam tm | TTpAll tm
pf ::= Gpa h | Hyp i | Known h | PTmAp(pf,tm) | PPfAp(pf,pf)
```

```
    | PLam(tm,pf) | TLam(tp,pf) | PTpAp(pf,tp) | PTpLam pf
```

plus `check_doc` (the proof checker), $\beta\eta\delta$ normalization with resource limits (`BetaLimit`, `NonNormalTerm`), and hash-addressed `doc`/`theory`/`signa` imports. Logic and set-theoretic content are *not* in this core: `True`/`False`/`not`/`and`/`or` are impredicative definitions, and membership/extensionality/choice/$\in$-induction enter as *axioms* in a preamble (Egal mode uses `Eps` choice and Tarski's axiom; Mizar mode uses `the` and ZF-style axioms). The seed file `Mettapedia/megalodon/experiments/01_basics.mg` proves eight propositional theorems entirely by proof-term lambdas (e.g. `andEL := fun A B H => H A (fun a b => a)`).

2.3 Precedent that MeTTa can host a checker

`hyperon/metamath/pverify` (with `lib/pverify-utils.metta`) is a Metamath proof checker realized as a MeTTa program. It establishes feasibility of the *host*; it is Metamath, not HOTG, so it is precedent, not evidence about the present design.

3 Architecture: the kernel as a MeTTa program

Shared waist (written once in MeTTa). Atom encodings for binders (de Bruijn or locally nameless), α-safe substitution, typed contexts, declarations (parameter / definition / axiom / theorem / inductive / import), proof objects, the **conversion engine** ($\beta/\eta/\delta/\iota$ as explicit, terminating, resource-capped MeTTa functions), a `check`/`infer` driver, and hashing/provenance for imports. The two checkers below are *instances* of this waist — that sharing *is* the early unification.

HOTG checker. Mirror `tp`/`tm`/`pf` + `check_doc` + $\beta\eta\delta$ as MeTTa atoms and functions. Proof-term checking over impredicative `Prop`; classical/set/TG axioms are a *profile* layered above the core, admitted only after the bare proof-term checker is green.

DTT checker. Mirror `PureTm` (`U0`/`U1`, Π, Σ, `Id`) with bidirectional infer/check, $\beta/\delta/\iota$, and decidable definitional equality by normalization; inductive families via `DeclSpec`, never hard-coded. Where the CeTTa engine's `-profile he-prime` already provides dependent type inference, the MeTTa kernel may use it; new C features are added to `he-prime` *only* when the prototype proves a concrete need.

4 Curriculum: the examples that pin the smallest fully expressive kernel

The curriculum *defines* "fully expressive": the smallest kernel is the one that checks every example below. The DTT ladder follows the chapter order of Smolka & Brown's *Introduction to Computational Logic* (a Coq course by Megalodon's own author), which introduces kernel features in increasing dependency order. The HOTG ladder follows Megalodon.

L0 — shared waist (both ladders ride on it). Names/binding, α-safe substitution, typed contexts, declarations, proof objects, conversion ($\beta/\eta/\delta/\iota$, decidable), hash imports + provenance, negative tests.

Table 1: DTT ladder (ICL spine + CeTTa `he-prime` seeds + `PureKernel`).

Step	Example (source)	Kernel feature it pins
D1	`bool/negb/andb/andb_com` (ICL §1.1)	inductive `Bool`, match/recursor, def-eq by computation
D2	`plus_O/plus_S/plus_assoc` (ICL §1.3–1.4)	`Nat`, recursor, structural induction
D3	Π / proofs-as-programs (ICL §1.2, 2.2, 2.6)	Π formation/intro/elim, application
D4	Σ / pairs (ICL §6.6; he-prime `TutPackHuman`)	Σ formation/intro/projection
D5	`Id`/refl/cong/transport (ICL §3.3; he-prime `TutEq/TutRefl/TutSuccCong`)	identity types, J/transport
D6	`app_assoc/length_app` (ICL §8)	parametric inductive + recursor
D7	`Vec`/`TutIndex` safety; **`VecN -1` (open)**	indexed inductive families; index-domain question (`Number` vs `Nat`)
D8	`Even`/$\leq$ (ICL §7)	inductive predicates, elim restriction
D9	recursors + elim restriction + universes (ICL §5)	ι-reduction, proof irrelevance, universe consistency

Table 2: HOTG ladder (`01_basics.mg` + Megalodon + `mathdata tp/tm/pf`).

Step	Example (source)	Kernel feature it pins
H1	propositional encodings (`01_basics.mg`, 8 thms)	`tp(Prop)`, `tm(All,Imp,Lam,Ap)`, `pf(PLam,PPfAp,Hyp)`, β-normalization, `check_doc`
H2	∀-instantiation; →/proof application	`PTmAp`, `PPfAp`
H3	proof λ; $\beta\eta\delta$; definition unfolding	δ on `ThyDef`/`SignaDef`, η
H4	equality/congruence (Leibniz `eq`)	polymorphic `eq` via `TpAll`
H5	**membership/set/TG axioms** (`In`/`Empty`/`EmptyE`, set-ext, `In_ind`, `Eps`)	*axiom profile*, admitted only after H1–H4 green

Cross-profile smoke. $P \to P$; $P \to Q \to P$; $P \wedge Q \to P$; $\bot \to P$; equality reflexivity/congruence; pair projections; and *one* tiny CeTTa operational-semantics theorem treated as a *certificate*, never as definitional equality.

Negative corpus (a fail-by-default gate — the soundness backstop). Bad de Bruijn index; unknown `Known`/`Hyp`; application type mismatch; a proposition-shape used as a proof object; cyclic/non-normal δ; **an arbitrary MeTTa rewrite admitted as conversion** (the central danger); stale import/hash; `VecN ...-1` wrongly accepted. The kernel must *reject* every item here; a checker that accepts any of them is unsound.

5 Prototype-first build plan

Phase 0. Shared MeTTa waist + the ledger harness (three deciders: MeTTa-kernel vs. Lean `PureKernel` reference vs. `he-prime` C).

Phase 1 (HOTG prototype). MeTTa `tp/tm/pf` + proof-term checker + β; *check the eight `01_basics.mg` theorems green* (H1–H2).

Phase 2 (DTT prototype). MeTTa `PureTm` + bidirectional checker + $\beta/\delta/\iota$ + decidable def-eq; *D1–D5 green*; ledger vs. the Lean reference authority and `he-prime` C.

Phase 3 (unify + harden). Both checkers share the waist; cross-profile smoke green; *negative corpus rejected*.

Phase 4 (climb the ladders). HOTG H3–H5 (definitions, equality, then set/TG axioms); DTT D6–D9 (lists, `Vec`, inductive predicates, recursors/universes); resolve the `VecN` index-domain question.

Later. Only after a fully working prototype and its lessons: consider deeper substrate integration — add the `he-prime` C features the prototype proved necessary; refine CeTTa. Not before.

Milestone 1 (the ledger, not a constitution). The deliverable is the working H1 + D1–D3 MeTTa kernel plus a populated *capacity ledger* with buckets Agree-Accept, Agree-Reject, Diverge, Open, plus the negative-corpus rejections. Divergences and open cases are first-class outputs, not blemishes to hide. Only then does external review become useful, and its question is whether the fragment boundary is honest and which divergences are bugs versus legitimate language semantics — not whether the final kernel is settled. The MeTTa-kernel verdicts are labelled **subject**; Lean `PureKernel` and Megalodon are the **calibration authorities**.

6 Realized architecture and results (June 2026)

The prototype is built and runs. This section records the *as-built* kernel and, honestly, where realization *refined* the plan above; the earlier sections stand as the original intent.

One $\lambda\Pi$-modulo checker; object logics as signatures. Rather than two separate term languages (`PureTm` and `tp/tm/pf`), the realized kernel is a *single* $\lambda\Pi$-style logical-framework checker (`Srt/Con/Var/Pi/Lam/App/Def`) with one `infer/check/conv`; each object logic enters as a *signature* of typed declarations over that one checker. DTT, HOTG, and HOL all appear in `kernel/curriculum_lf_demo.metta` as signatures over the same checker, each with positive proofs and a rejected negative. This keeps every logic's rules first-class (as signature declarations) while sharing the trusted waist — LF-hosting, deliberately distinct from desugaring every logic into one ambient rewrite theory.

One generic inductive engine (`DIndG`). The hard-coded inductive families of the DTT ladder (D6–D9) are subsumed by a single generic *indexed-inductive* declaration, `DIndG`: a telescope IR (parameters Δ, indices Ξ, constructors) from which the family kind, constructor types, the eliminator, branch types, and constructor-headed ι-rules are *generated and checked*. `Nat`, `Bool`, `List`, `Tree`, `Pair`, `Vec`, `Fin`, and `Id` are all instances; crucially `Id` is itself a `DIndG` and `J` is its *generated* (diagonal) recursor, so the equality primitives and six special-case inductive slices retired onto the

one engine. (This resolves the original `VecN`/index-domain open question: indices are an arbitrary telescope, not a single `Nat`.)

Binding as a declared property (the ABT waist). Binding lives in one import-free waist (`kernel_binding_waist_v1`): de Bruijn under the hood, with capture-avoiding `shift`/`subst` written once and *arity-parameterized*; clients delegate to it rather than re-implementing substitution. A `BindingDecl` layer (`kernel_binding_decl_v1`) lets each hosted LanguageDef *declare* its binders (`Bind0`/`Bind1`/`BindFields`/`BindPi`: dependent Π, ∀/∃ bodies, comprehension predicates, proof binders) without disturbing the hot waist. Capture-avoidance is mutation-gated.

Admission, checked-once theorems, and the admit-once lesson. A signature is *admitted* only if every declaration is well-formed: constants' types check, definitions' bodies check against their declared types (`wf-decl`), and inductive declarations are fully generatable. Two declaration forms serve derived libraries: `DAxiom` (an honest postulate, recorded in *axiom provenance*) and `DThm` (a *checked* theorem constant — its proof body is checked *once* at admission, then used opaquely by its type, transparent only for provenance). A memory hazard forced the central engineering lesson: naively, `kernel-check` re-ran full signature admission on *every* theorem check, so a file checking 21 theorems against a signature carrying two large derived-proof bodies re-normalized those bodies 21 times and exhausted memory (∼48 GiB, OOM). The fix is *admit-once batch checking* — admission computed once, lexically, for a batch of jobs, with *no* forgeable "checked-signature" token exposed (in MeTTa a token is plain data anyone could fabricate). This is exactly HOL-Light's discipline (a derived rule is a program calling primitives, never one giant proof term re-normalized from scratch) and it dropped the same file to ∼8 GiB.

Faithful object-logic hosting: HOL-Light's equality kernel. HOL is hosted not as a convenient natural-deduction surrogate but on HOL-Light's *actual* primitive basis — `REFL`, `TRANS`, `MK_COMB`, `ABS`, `BETA`, `ASSUME`, `EQ_MP`, `DEDUCT_ANTISYM` as LF rule certificates checked by the imported kernel — with ⊤/∧/⇒ defined over equality and λ (matching `bool.ml`) and derived `MP`/`DISCH` shards admitted as checked `DThm`s. Each primitive and each shard carries a mutation tooth. *Faithfulness caveat:* the definitional equations are currently *postulated* as constants matching `bool.ml` rather than introduced through a checked conservative-definition principle, and the derived `MP`/`DISCH` are specific checked replays, not yet a fully general derived-rule library.

The curriculum as a calibration matrix. The curriculum is now an explicit ledger in the `Curriculum` directory: each row is a kernel capacity × profile (DTT/HOL/HOTG/Metamath/...) × the SOTA system that calibrates the verdict (Lean/Coq/Agda/Megalodon/HOL-Light), with status CHECKED or PLANNED and a per-row gate. The external systems are the *calibration authority*; the destination is to *host each core logic as a LanguageDef and re-prove the curriculum's theorems in our own kernel*. Verdicts are differential — a row borrows its expectation from the source system, not from our own guess.

The Metamath profile (August 2026 update). The Metamath row of the matrix is now the furthest along the stated destination. The full statement language is authored twice at the `LanguageDef` level — once in `languageDef!` form (`metamathCore`, including the front-end lowering pipeline as rewrite rules) and once as the proof-facing `sourceGrammar`, whose presentation passes the generic validator and whose accepted pipeline runs derive the outer database sort in the lexicalized language of their own classified streams with no supplied lexical

premises (`runSource_selfLexicalized_derives`). On the semantic side, every accepted source declaration transition is matched by the corresponding shipped `mm-lean4` database operation, and the whole statement fold co-evolves with the implementation database from the initial states (`default_initial_applyPayloads`), including the identification of the implementation's mandatory-frame trim with the specification's mandatory frame (`trimFrame'_eq_mandatory`). Checked source admission runs exact bytes through the actual verified reader into one generated presentation package. Deliberately open, in the obligations ledger's sense: per-`$p` proof discharge from the reader's own chronology (normal lane recently constructed; compressed in progress), the distinction between verified and `?`-incomplete outcomes as disjoint results, and the prefix-wise source/reader simulation in which reader microsteps are administrative. The Metamath lane thereby serves as the template for how the other profile rows are intended to mature.

Validation discipline. Every checkpoint runs on three engines — CeTTa (C), PeTTa (Prolog), and the Lean-verified *LeaTTa* as a deterministic reference — in parity, with per-feature mutation gates (corrupt the rule, require red) and the fail-by-default negative corpus, all behind one `run_all` gate; heavy runs are memory-capped.

Soundness status (honest). Admission plus provenance close the per-input declaration holes (a `break_probes` probe documents what the bare term-checker would accept *without* admission; the real kernel rejects those). The *one* genuinely-open soundness obligation is metatheoretic, not a per-input check: **universal subject-reduction, confluence, and termination of the generated ι-rules are not yet machine-proven** — we have an executable SR-wedge self-test and the LeaTTa engine-floor, not the theorem. That metatheorem is the live Lean target. The kernel is today *checked-correct per input* but not yet *proven-correct as a calculus*.

7 From one hosted checker to a proof-aware NIK and native MeTTa theory

The runnable logical-framework kernel above motivated a more general theory, but it is not the definition of that theory. Three concerns remain distinguishable for derivation and audit:

```
NIK       laws of authorities, evidence, admission, and transport
MIK       MeTTa realizations of selected authority laws
guests    native MeTTa DTT, Pure, Megalodon/HOTG, Metamath, HOL, ...
```

This list is a decomposition of obligations, not a vertical system architecture or a sequence of foreign services. An authority fixes a claim family, its intended meaning, and how acceptance is established. A realization computes or checks that acceptance. The guest fixes the object logic. Consequently NIK has a metajudgment about accepted evidence, but no privileged object logic. The native MeTTa type theory is both a guest and the intended language in which language definitions, translations, proof operations, and lower-level authorities are represented as ordinary typed values. Megalodon and `PureKernel` remain independent calibration authorities rather than components silently baked into the NIK waist.

The first declaration-aware Prime root now tests this separation formally. Its ordinary typing, context-formation, and universe-formed typing judgments inhabit one disjoint exact codec, and one proof-relevant interpreter covers the resulting presentation compositionally. The formed judgment is a conservative derived rule with three ordered premises—formed context, formation of the expected type, and inhabitation by the subject—rather than a second typing relation. Generically,

universal semantic fibres turn such a disjoint coproduct of judgment codecs into the product of the independently usable semantic views; retained rules acquire a new vacuous fibre precisely when none can conclude a wire owned by the new judgment family. Accepted raw trees therefore reconstruct intrinsic evidence at ingress, while the resulting displayed type and term contain no checker. The important host lesson is structural: another logic supplies its own judgment indices and semantic fibres, while the reusable waist supplies exact encoding, validated extension, proof checking, interpretation, and currentness. This result does not yet constitute complete Lean, Coq, or Megalodon hosting; their native conversion and declaration disciplines remain separate obligations.

The source of a hosted calculus is now separated from its extensional lookup environment as well. An authored declaration document elaborates exactly and in order, retaining complete equation schemas and a stable occurrence index for each instantiated equation. Its semantic interpretation deliberately uses the existing first-wins constant signature and proof-erased computation support. Two distinct source inventories can therefore have the same semantic signature, while duplicate equations with identical endpoints still have distinct native receipts. This noninjectivity is a boundary theorem: an optimized environment is a useful interpretation of a source program, not a reconstruction of it. Moreover, mapping a retained equation occurrence merely to the proposition that some occurrence exists is proved noninjective. Logical support is therefore a valid conversion readout but not a receipt format.

Intrinsic typing is consequently indexed by an authored declaration host. A formation host proves that every declaration is meaningful; a computational host additionally proves that its authored equation steps preserve typing. Every judgment in the bare checked fragment embeds into every formed extension, and a formed declared constant receives an intrinsic hosted typing judgment that the empty host cannot manufacture. Formation alone does not grant a conversion algorithm, normalization, confluence, or native execution. Those are separate capabilities to be earned by each selected family or guest.

Nominal ingress is source-faithful rather than a bare membership test. A proof-relevant first-occurrence witness walks the ordered declaration document, retaining skipped equations and differently named constants, and stops at the first constant with the requested name. Its stored index reads back the exact source declaration. This witness exists if and only if the extensional first-wins signature lookup returns that complete entry; hence preservation and the strongest true reflection hold at this boundary. The witness directly constructs the declaration-hosted typing judgment, while a later shadowed declaration is proved unable to acquire the same authority. A generated raw checker and a native constructor may therefore consume one common source fact without either recovering provenance from an optimized signature or placing checking inside native construction.

This embedding now crosses the checked/native waist itself. The same fixed formed-typing presentation and exact raw proof tree can be interpreted in any authored formation host; its native artifact is the corresponding hosted context, type, and term judgment. The reverse inclusion is false in a useful way. For every actually declared constant, the hosted calculus constructs a typing inhabitant while the fixed bare checker image at the identical surface claim is empty. Thus a source-generated checker extension may admit raw declaration syntax, but it must not be mistaken for the source of native declaration construction. Generic checking is the common ingress floor; host-specific introduction and computation remain strictly stronger earned faces of the same judgment.

For the current native List and identity package, the source-to-host direction is now exact. One authored document contains the five constant declarations and three typed iota schemas. Its ordered GSLT elaboration interprets to the existing formed signature. More strongly, at every pair of endpoints its indexed equation-occurrence carrier is equivalent to the native iota carrier: the equivalence retains schema identity, the complete authored substitution, and every native eliminator argument in both directions. A generic source-directed presented candidate now requires this fibre-

wise equivalence; equality of logical support follows as a theorem and is not accepted as a substitute. Promotion of the same package from a formation host to a computational host still requires the independent preservation family. A raw authored nil reduction with an undeclared element supplies the negative control: exact receipt agreement does not manufacture a typing derivation. Nothing here claims that an extensional signature can decode its source. The construction is now generic over a finite authored inventory: declaration order, constant lookup equality, and the receipt-fibre equivalence derive the entire presented candidate and its interpretation theorem. A separate negative theorem rules out any total decoder which would recover every such declaration inventory from its extensional signature. The same construction is instantiated independently for a shared Nat/Vec declaration world. Eight constants and four iota schemas generate one authored source; Nat and length-indexed Vec are distinct strictly-positive candidate views of that source, and both views carry the same exact receipt-fibre equivalence. This demonstrates that a presentation is a declaration-world object rather than one opaque registry record per family.

The authored/native agreement now meets intrinsic typed conversion constructionally. For a presented candidate P, define the fibre of typed occurrences by

$$\mathsf{TypedOcc}_P(\Gamma, l, r, A) = \mathsf{Occ}_P(l, r) \times (\Gamma \vdash l : A) \times (\Gamma \vdash r : A).$$

The receipt-fibre equivalence sends such an occurrence to one exact native step between typed states, hence directly to proof-relevant conversion evidence. A second, explicit map forgets the receipt to the existing proposition-supported declared conversion. The native step still recovers the authored occurrence exactly; the support projection is intentionally not claimed faithful. If the declaration package proves uniform preservation, an authored occurrence and the source typing construct the whole fibre, with the target typing derived once from preservation. Canonical List-nil and Nat-zero reductions instantiate the construction, whereas corresponding raw reductions with undeclared parameters are proved to have no typed occurrence.

Pointwise receipt equivalence is not yet enough for dependent computation: its comparison must also be natural in the open telescope. The native-family interface therefore exposes this as a separate earned capability. For every renaming or simultaneous substitution σ, the square

$$\begin{array}{ccc} \mathsf{Occ}(l, r) & \xrightarrow{\eta} & \mathsf{Native}(l, r) \\ {\scriptstyle \sigma}\downarrow & & \downarrow{\scriptstyle \sigma} \\ \mathsf{Occ}(\sigma l, \sigma r) & \xrightarrow{\eta} & \mathsf{Native}(\sigma l, \sigma r) \end{array}$$

commutes on the exact proof-relevant receipt, not merely after support erasure. Formally, authored occurrences and native receipts are loose arrows on open terms; the fibre equivalence gives mutually inverse equipment cells, and the displayed square is an equality of their two vertical composites. Consequently a typed occurrence transports along any typed context morphism, and translating its transported authored witness yields exactly the receipt obtained by native substitution. The theorem is proved for List/identity and for both Nat and Vec views of their one shared declaration source. The ill-typed nil and Nat-zero occurrences remain outside every typed fibre, so naturality does not manufacture authority. At the GSLT-IL layer this square is the required operational cell between source transport and native realization; it does not require a global native-family conversion checker or force a relational elaboration to become a function.

This naturality law supports a stronger constructional kernel without making the raw relation stronger. A typed native presentation supplies one canonical typed occurrence for every authored equation schema and proves that it retains that exact source position. This presentation is now generated from one source-wide finite inventory containing both endpoint typings at every authored schema position, together with the receipt-naturality law. The scope is important: the inventory

ranges over the whole declaration world, not one family's local iota registry. In the shared Nat/Vec source, each local registry has two clauses while the authored source has four schemas; nevertheless the Nat candidate view constructs the Vec-nil step at its source position. Thus local family coverage is neither silently promoted to source coverage nor used as a dispatcher boundary.

The kernel generated from this presentation has typed states as its indices. A primitive step consists of a schema position, a typed simultaneous substitution, the endpoint equations, and an explicit displayed-type adjustment. Hence both endpoints inhabit the same typing fibre by construction. Canonical steps and all their typed substitution instances are generated directly, and a structural map sends complete reflexive, symmetric, and transitive conversion trees into the existing proof-relevant native computation while preserving every intermediate state and association node. Native List/identity supplies one instance. The shared Nat/Vec source supplies two more candidate views over the same four-schema declaration world, showing that generation is attached to a typed presentation rather than to a hard-coded family dispatcher. The raw nil and Nat-zero equations with undeclared parameters supply the negatives: each equation remains available to raw execution but cannot be the source of a constructional step. Extending this kernel to every syntactic substitution in the raw equation closure still requires the independent preservation or conversion-coherence theorem. Thus the constructional face is automatic on its typed image, while raw computation authority is neither assumed nor lost.

This bridge does not turn the generic checked presentation into an interpreter for every native computation. Its present typed-conversion checker describes the diagonal boundary fragment, while native beta and iota realizations construct richer steps in the same judgment-indexed semantic carrier. Thus an authored family may itself be quoted and transported as GSLT-IL data without requiring its already-admitted eliminators to replay that presentation during execution. Reflection describes and transports the calculus; it does not replace its native computational structure.

The same comparison now reaches NIK as a constructional admitted flow. At one fixed judgment index, a typed authored occurrence and a complete native receipt are equivalent: the forward map retains both endpoint derivations and the exact native computation witness, while the inverse recovers the authored occurrence. The two composites are identities. Storing the forward map at a dependency revision therefore authorizes a certificate-free active operation whose runner only constructs the native receipt; it neither replays the source presentation nor invokes a conversion checker. Current execution is exactly the forward GSLT-IL equipment cell and commutes with typed substitution. A changed selected dependency prevents activation without erasing the typed source, and a raw ill-typed equation remains expressible below the boundary but cannot inhabit the admitted source fibre. List, Nat, and Vec reductions instantiate the same generic bridge. This is the maximal-native principle in its constructional case: reflection supplies the semantic comparison, typing supplies the inhabitant, and NIK retains the strongest justified no-replay realization.

There is nevertheless a strict upgrade above this constructional face. Put a typed source generator and a raw source-typed authored equation into one gradual request language. The constructional operation realizes the former and returns the latter unchanged for ordinary fallback. A declaration-wide preservation theorem constructs the missing target judgment for the raw request, so a second operation realizes both forms without replay. These operations share one proof-carrying result contract: success retains target typing, native evidence, and exact authored provenance; fallback retains the complete raw request. Capability inclusion orders constructional image below uniform raw preservation, and the latter family cannot even be formed without the preservation theorem. NIK consequently selects uniform preservation as the unique strongest member when that capability is requested, while a family that has proved only its typed source image remains honestly constructional. This is a useful implementation rule: optimize the exact typed image now; only expand the no-replay boundary when the independent preservation proof lands.

The constructional face now also instantiates the common lazy gradual law without adding another evaluator. Its exact evidence is displayed over the complete raw source request and retains the judgment index, both typed endpoint states, the proof-relevant constructed step, and equality of that step's erasure with the request. Exact current evidence enters the no-replay native operation. Suspension or a local failure to establish this optional evidence enters the ordinary fallback face with the identical request; neither is a semantic rejection. Revision staleness forgets only the optional construction witness, and a changed dependency revision cannot activate the cached admitted operation. Thus precision changes the justified execution face while leaving the authored source, context, type, occurrence, and fallback semantics fixed.

Typed contextual substitution acts through that same displayed boundary. A transport request carries the target telescope, simultaneous substitution, and its context-typing derivation. On exact evidence it composes the authored schema substitution, transports both endpoint derivations and the directed type adjustment, and constructs another native step. Erasing this transported step gives exactly the raw request obtained by substituting the original erasure. Raw request transport satisfies identity and composition, so this is contextual structure rather than a family-specific helper. The induced gradual map preserves exact evidence, preserves suspension, and invalidates unsupported blame to suspension; revision invalidation commutes with it. A nonidentity weakening of the List-nil context realizes natively when its construction witness is retained and follows ordinary fallback when the same transformed request is suspended. This theorem concerns typed contextual transport within one hosted declaration world; a translation between distinct authored family worlds must separately supply its declaration and receipt map.

The construction extends from generators to complete conversion derivations by a general initiality theorem, rather than an indexed-family-specific recursion. For any judgment-indexed state and step families, interpretations of primitive steps, reflexivity, symmetry, and binary transitivity into an indexed target family induce a unique structural fold on complete conversion evidence. The type of constructor-preserving extensions is contractible. Consequently an exact equivalence on every primitive step fibre lifts to an exact equivalence of complete conversion trees. Instantiating this theorem, the authored and native indexed-family systems retain every intermediate typed term, occurrence identity, native receipt, symmetry, and composition node in both directions, and the lift remains natural under typed substitution. NIK may therefore admit one structural path translator and execute it directly at the current revision. This result is intentionally prior to higher coherence: a witnessed round trip is distinct from reflexivity, and the two association trees for the same three legs remain distinct even though their endpoints agree. Endpoint-only normalization, caching, or replay would erase information that the native calculus still exposes; any later identification of these paths must be supplied by an explicit higher cell.

The complete lift also inhabits the same lazy gradual discipline as primitive construction. The raw value is the entire authored conversion tree; exact evidence is a native tree together with equality to its structural lift. Typed contextual substitution maps both trees, and receipt naturality proves that the transported native evidence is exactly the lift of the transported authored path. Exact evidence therefore remains exact, while suspension and unsupported local blame retain the complete authored tree for fallback. Revision staleness forgets only the optional native realization. For a nonidentity weakening of the List context, the exact branch preserves the whole three-leg association tree and agrees with NIK's current selected complete-path result; the same weakened path without evidence follows fallback. Because every valid authored path has a structural native lift, a sound refutation of that lift is uninhabited. This closes the gradual conversion-path square without quotienting receipts or adding a checker.

This universal lift now enters maximal-native selection as an explicit capability rather than a fixed mode priority. Over one gradual path-realization contract, a primitive face realizes a single

generator and retains every composite authored path for fallback; a complete face realizes the unique structural extension and carries a proof that the inverse lift recovers the entire authored tree. Capability inclusion makes the latter a strict upgrade, so a request for complete proof-relevant paths has one unique strongest member. The selected current runner is definitionally the free-conversion lift. This does not globally rank all NIK modes: native proof construction and explicit boundary replay still have no greatest member when the request leaves their incomparable boundary capabilities unspecified. Hence "use the strongest kernel" is a theorem exactly where a common semantic contract and a directed capability fibre justify it, and a maximal frontier everywhere else.

This free conversion algebra is deliberately not identified with the neighboring free directed-path category used for GSLT execution. The latter composes paths associatively; the former still exposes the syntax tree of reflexivity, symmetry, and transitivity and assumes no unit, associativity, cancellation, or involution equations. Iterated identity types can support weak ω-categorical or weak ω-groupoidal coherence, but that additional structure must be constructed and connected to these retained receipts. It is not obtained by silently quotienting them.

The same distinction now holds at the displayed gradual boundary. Raw judgments are indexed by suspended, exact, or refuted evidence, and every constructional map acts on raw syntax and exact evidence while invalidating negative evidence unless reflection has separately been earned. Commuting squares lift this action to gradual states. Lambda, application, pairing, projections, identity formation, and reflexivity commute with typed context substitution; direct Pi-beta and first-$Sigma$-beta additionally retain the same proof-relevant structural receipt on both routes. Native identity-iota cells enter this exact fibre from their intrinsically typed declaration receipt, whereas changing the authored target is uninhabited.

These constructions now satisfy one compositional lazy gradual-dependent guarantee rather than a list of isolated examples. Forward transport of exact evidence composes strictly. Since a merely forward map need not reflect exactness, its identity action may deliberately forget cached blame; once an exact-reflection capability is supplied, transport preserves blame and becomes fully functorial. Revision activation commutes with both forms of transport, with independent products, and with genuinely dependent sums. Hence a current typed constructor may run without an interior check, while a stale capability returns to suspension without changing its raw term, path, or program. Every proof-relevant naturality square acts on all three gradual states, and the ordinary runner for a checked plan has no checker argument; first demand evaluates once and subsequent cached observations are inherited from the shared Need semantics.

Whole programs and proof-relevant rho worlds are no longer exceptions to this displayed construction. A complete authored program is the raw index and a source-preserving plan is exact evidence over precisely that source. Promoting an island is therefore an exact map whose raw action is identity; matching typing evidence changes the selected plan, while independently valid evidence for another candidate cannot cross-promote it. Dually, an already-authorized rho branch is the raw index and a certified finite-family schedule is exact evidence over that branch. Current evidence realizes the native schedule; suspension, local blame, or revision staleness realizes the exact raw branch. Two competing communications inhabit different raw indices, so gradual precision cannot silently select, merge, or serialize them. NIK may choose the strongest current evidence within one fibre, but nondeterministic choice remains part of the language semantics rather than an artifact of optimization.

This distinction is strict, not terminological. There is a constructional many-to-one map which preserves every exact source witness but cannot reflect exactness at an unsupported source point. It therefore earns the complete forward-safe law package and provably cannot earn stable-blame transport. Conversely, identity transport earns reflection and retains blame. These are requestable

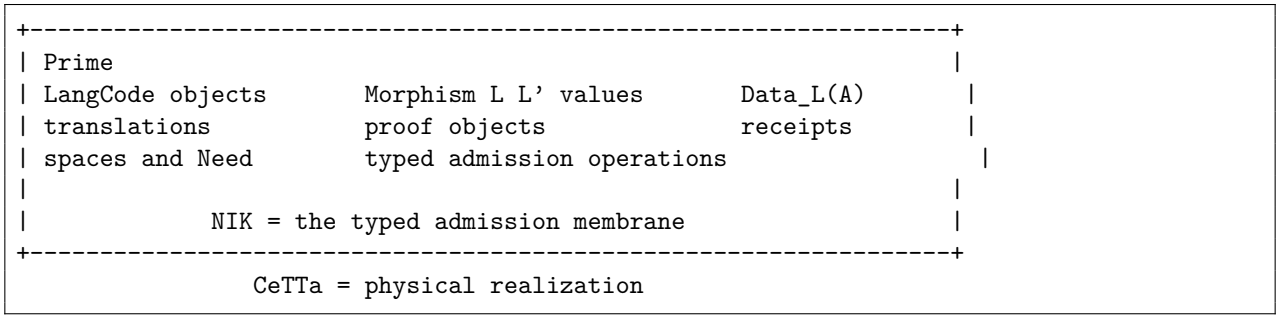
native-calculus capabilities: a maximal implementation may select the reflecting face when its guest proves reflection, but must select the forward-safe face or deoptimize otherwise. This is now an instance of the request-local maximal-native theorem rather than an implementation convention. Every exact map has a unique strongest member in its safe registry. Registering an explicit reflection witness and its derived laws adds one strict common upgrade, which is the unique strongest member for both exact-state and stable-blame requests. For the non-reflecting collapse, the stronger registration and every nonempty stable-blame request are uninhabited. A revision-current NIK admission runs the retained selected operation directly, while a relevant dependency change prevents activation and the gradual state returns to suspension without changing its raw index. No global ranking of checkers and kernels is needed, and failure to earn the stronger face removes no raw behavior.

Dependent second-*Sigma*-beta reveals the first genuinely higher coherence obligation. Its source and target types are connected by a conversion receipt and its terms by a separate computation receipt; the two type codes need not be equal. Ambient substitution commutes on the complete raw boundary and on the term receipt. The type-conversion receipt, however, is generated pointwise from the codomain, and substituting a compound term for one variable may expand its proof tree. The formal interface therefore retains both conversion trees and does not identify them by proof irrelevance. A comparison between them must be an explicit higher cell (or a stated quotient with an adequacy theorem). This is not needed to execute the native eliminator, but it is required before claiming a fully coherent bicategorical or homotopical semantics of all retained receipts.

Typed conversion supplies the next bridge without imposing one conversion algorithm on every guest. One exact judgment index records the formed context, the common type and universe, both endpoints, and a proof-relevant conversion path. The generic presentation currently earns a diagonal fragment: from an accepted formed typing judgment it derives reflexive conversion, and every derivation in that fragment is proved to have equal raw endpoints. Prime's native dependent kernel nevertheless constructs a nontrivial well-typed Π-beta path in the same intrinsic family; its endpoints are syntactically unequal, so it is formally outside the reflexive checker image. Conversely, a raw beta step that erases an ill-typed argument has no inhabitant of the typed conversion family. Thus there is one semantic wire with several earned native realizations, rather than either one universal conversion algorithm or several unrelated notions of judgment.

This is also the intended path for Lean, Coq, and Megalodon integration. The hosting layer can be reused for exact claim codecs, source identity, checked proof trees, proof-relevant interpretation, revision currentness, and admitted transport. Each guest must still contribute its own declarations, universes, conversion, inductive principles, and trust boundary, together with the strongest honest preservation and reflection theorem for its represented fragment. Sharing the waist therefore reduces integration work without pretending that CIC conversion, a higher-order set-theory kernel, and Prime's native computation are interchangeable.

The target system boundary is therefore one reflective Prime, physically realized by CeTTa:



At the present formal boundary the language objects, structural language arrows, and their

covariant Data action have semantic representatives as dependent terms in the selected Prime CwF. Syntactic definability of the whole GSLT-IL operation fragment and internal admission operations remain open. The diagram is the integration criterion, not a claim that those remaining theorems are already proved.

7.1 Dialects are points in a specification region

The names Zero and Prime no longer denote immutable semantic records. The Lean development separates a region of admissible operational points from the current branded point. Zero's region consists of occurrence-indexed operational points with bag observation attached separately. Prime selects an operational base, a bag observation, and a revision authority; the present query-first Zero backend is one assembly. Need, reflection, proof hosting, and the authored presentation are independently attached layers. Typed translations relate the current Zero and Prime points, and negative theorems show where those arrows cannot be strengthened: reflection contributes steps absent from the base, Need can escape the base image, and the Need route cannot in general be erased back to occurrence evaluation.

The native-type-theory derivation uses two inputs which must not be conflated. An *observation bag* records properties of a current dialect point: collection results, metavariable patterns, native reflection, first-class spaces, inert unknowns, costs, evidence, or staging. A separate *target-commitment* channel declares capacities the future family is required to gain: dependent-family reasoning, identity elimination, typed language manipulation, native proof-object programming, stratified bootstrap, and a direct certificate-free typing path. The theorem `commitments_extend_observations` proves that the commitments are not being laundered as observations of today's Prime. The direction of fit is therefore explicit: Prime may change to meet the commitments; the theory is not weakened to describe only the current implementation.

The current bags remain auditable authored data. Connecting each entry to a live dialect capability witness and an exact runtime parity theorem is a remaining realization task. Until then, the bags guide derivation and reject brand-fixing, but are not an automatic extraction of the CeTTa implementation.

7.2 The derived MeTTa-native type theory

The formal spine is a contextual, multimodal, intensional dependent type theory. It contains a category-with-families style context/substitution discipline, Π- and Σ-types, identity with J, dependent and binary inductive families, noncollapsed Tarski-style universes, typed quotation through explicit locked contexts, level-indexed staging, cost grading, proof-relevant evidence, and first-class codes for language presentations. Types-as-spaces is represented by an interpretation into predicates on Patterns with an explicit subtyping order; cost and evidence remain finer decorations and are not erased into mere truth sets.

Equality follows a proved profile architecture:

- The initial core has intensional, computational conversion.
- Behavioral equality is an explicit modal proposition or named profile, not silent kernel conversion.
- Function extensionality is not judgmental in the core.
- K/UIP is declared per profile.

- Cubical structure and univalence remain possible future authorities and do not burden every core judgment.

The present strictly-positive indexed-family mechanism is therefore a precursor to, not a substitute for, higher inductive types. A computational homotopy profile must additionally provide dimension contexts, face constraints, composition or filling operations, path constructors with boundary data, and canonicity for the selected fragment. Indexed cubical inductive types give a particularly close precedent because they combine ordinary indexed constructors with higher-dimensional constructors and computational rules. Such a profile should extend the native family presentation and receipt fibres; it should not silently identify authored operational histories or impose univalence on every guest language.

A concrete observational profile is proved to be a strict extension of the core; old derivations transport into it, the reverse inclusion fails, universes remain distinct, and an anti-leak theorem prevents tag erasure from turning profile equality into core conversion. The small Boolean profile used in this proof is a calibration witness, not the selected production equality algorithm.

The named audit object now has a concrete nondegenerate witness for every required field and its `openObligations` list is empty. This closes the abstract derivation audit, not the entire implementation project. The executable `FastPatternTyping` authority is a direct, certificate-free fragment over the live PeTTa `MeTTaType`; the full proof-relevant native typing doctrine has assumption, weakening, substitution/cut, evidence multiplicity, and proof erasure. What remains is one production-strength Pattern elaborator and conversion engine whose executable judgment has exact parity with that full declarative doctrine.

The status rule is deliberately memorable:

```
zero obligations = abstract audit, not production checker
```

Emptying the derivation ledger means that every field of the abstract candidate has a nondegenerate mathematical witness. It does not by itself supply a production elaborator, decide full Pattern conversion, or prove an implementation adequate to the declarative judgment.

7.3 Language-indexed `Data`: quotation, hosting, and dialect change

Quotation is not a synonym for the top type and not a hidden calling convention. The derived target separates three notions which older MeTTa dialects partially conflate:

$$\mathsf{PDynamic} = ?\mathsf{Unknown} \quad \text{(gradual unknown)}, \qquad \mathsf{Atom} = \top \quad \text{(inert value top)},$$
$$\mathsf{Data}_L(A) \quad \text{(held code of language } L \text{ and type } A\text{)}.$$

Gradual consistency is nontransitive (for example, $\mathsf{Number} \sim ?\mathsf{Unknown} \sim \mathsf{String}$ without $\mathsf{Number} \sim \mathsf{String}$), whereas subtyping through the inert top is transitive. Evaluation control is a separate modal axis. Spaces may select an evaluate-by-default or data-by-default ambient polarity; explicit quotation holds in either space, and the concrete `!` syntax is the counit which evaluates already admitted Data in either space.

The syntax keeps object variables and elaboration metavariables visibly distinct: `$x` is an object variable, `?a` is a solvable meta-level variable, and `?Unknown` is the distinguished never-solved gradual type. The inherited spelling `%Undefined%` denotes that same gradual type; it does not introduce a second type or a compatibility wrapper. Provenance such as "declared unknown", "widened after conflicting uses", or "introduced at a boundary" belongs to evidence or blame data, not to the kernel type lattice.

The formal type-code closure is deliberately non-idempotent: `Data (Data A)` is not `Data A`. Its explicit level strictly rises at each quotation, so code which writes code and the stratified bootstrap tower retain their stage distinction. In each selected language fibre, the concrete modality is the product comonad $S_L \times -$, where S_L is a stage/space/revision stamp, not an execution trace. The counit projects an already-admitted payload and comultiplication retains the stamp while requoting. A Boolean-stamp canary proves that comultiplication is not surjective, so this is not an identity modality in disguise.

A typed language translation maps base type codes, their inhabitants, and stamps, then acts through every nested `Data` layer. Identity and composition are proved for both type codes and inhabitants, and the associated covariant Grothendieck construction has canonical strongly cocartesian push-forward lifts. Consequently the currently proved structure is an opfibration-style family with chosen cocartesian transport; the stronger packaged claim "split opfibration" is withheld until its cleavage coherence is stated at that abstraction layer. The word "fibration" is used only informally for the family: no contravariant pullback, and hence no genuine fibration theorem, is claimed without additional structure. This abstract family is instantiated on the actual category of validated language definitions and structural GSLT morphisms: a fibre's base codes are exact authored sorts and its inhabitants are actual closed, well-sorted object terms at those sorts. Structural morphisms map both the sort and the held syntax while preserving its typing derivation; the resulting dependent family has functorial Data transport and the canonical strongly cocartesian lift for every authored route. Thus the index is not a two-language enumeration, an unstructured string, or a unit-valued witness which erases the program. The live two-point instance additionally uses the existing Zero-to-Prime operational translation on semantic-state type indices, preserves the indexed state exactly, has a strongly cocartesian promotion lift, and has no reverse Prime-to-Zero push-forward.

NIK governs entry into these fibres; it does not turn the counit into replay. Direct decisions, native proof kernels, and admitted flows all have no separate mandatory certificate language. Only an explicitly selected certificate boundary does. Once a value has entered `Data`, `!` does not inspect which admission mode established it. This is the formal separation between a trust membrane and ordinary typed execution.

The four inherited HE meta-types `Symbol`, `Variable`, `Expression`, and `Grounded` are represented as disjoint refinements of held syntax rather than as evaluation-changing supertypes. Their classifier is total and single-valued, all four refinements have concrete inhabitants, and a grounded hook is proved not to inhabit the expression refinement. This keeps the useful syntax taxonomy while removing its control-flow side effect.

The first authored dialect delta is also formal. In value position HE `Atom` maps to Prime's inert top; in an argument domain its historical hold-before-evaluation convention maps to `Data _`. Call-site elaboration is preserved exactly: the translated signature inserts the hold which HE obtained from the parameter type. The partial inverse is proved on precisely the image of the translation, and a negative theorem shows that Prime's gradual dynamic type has no fabricated HE preimage. Constructor translation changes constructor heads only; de Bruijn indices and free metavariable names are fixed pointwise, free-name sets are preserved, and well-scoped source syntax remains well scoped. More strongly, translation commutes with the canonical locally nameless opening, closing, lifting, and binder-eliminating substitution operations. Open code is now packaged as actual authored `OpenTerm`s indexed by an exact free-variable typing context, an ordered de Bruijn type stack, and an authored carrier sort. Language/context pairs and context-preserving structural routes form a category; their fibres contain these open terms, and their Data action is functorial on both type codes and inhabitants. A Zero-to-Prime witness transports a genuinely open Atom variable while retaining its name and mapped type. An out-of-range bound index is rejected

before quotation, and a target context which does not classify a free name cannot receive that name through transport. Thus the contextual box transports without weakening or capture; a nonidentity constructor translation remains the negative control showing that hygiene was not proved by making translation trivial. These are the mathematical gates for a future `cetta -trans he.prime prime` realization, not a claim that the command-line realization already exists.

Syntax hygiene contract. An open quotation is formed only from a judgment $\Gamma \vdash_L t : A$ and records L, Γ, and A in its contextual Data type; raw text or an untyped Pattern is not a quotation premise. Free object names are exactly those classified by Γ, while bound variables use locally nameless indices and canonical binder metadata. A language route $T : L \to L'$ transports the whole judgment to $T\Gamma \vdash_{L'} Tt : TA$: it changes authored constructors and types, but fixes free names and bound positions and commutes with opening, closing, lifting, and capture-avoiding instantiation. Splicing requires an explicit context map into the surrounding context; accidental name coincidence supplies no authority. The counit `!` applies only to already admitted contextual Data, and does not convert failed admission or incomplete guest checking into evaluation. Ambient space polarity may insert quotation or evaluation at elaboration boundaries, but it cannot alter these typing and capture conditions. The final syntax spelling may infer recoverable indices, but it must elaborate to this judgment and may not weaken it.

This construction is a specification ablation as well as a proposal. Collapsing `?Unknown`, `Atom`, and `Data` destroys the proved nontransitive gradual-consistency witness, makes a nominal upcast change evaluation behavior, and removes the separate held-syntax index. Making `Data` idempotent destroys the strict-level theorem. Removing language indices loses typed guest hosting and language-changing transport. Requiring a certificate at the counit contradicts the certificate-free direct and admitted-flow modes. The surviving design is therefore selected by independent positive and negative laws rather than by the spelling of today's dialect.

7.4 How GSLT/MeTTa-IL and Prime properties determine the syntax

The theories retain different jobs, but those jobs must not become different external products. A GSLT language definition supplies authored syntax, equations, and one-step dynamics. MeTTa-IL supplies an indexed language in which definitions, translations, and execution routes are themselves manipulable data. These objects and operations must be internal to Prime: users must be able to store, match, translate, mine, and chain over them with the ordinary Prime language. Prime also supplies the operational commitments which must survive that abstraction: open-world inertness, collection-valued answers, first-class spaces, revisioned Need, costs, evidence, and reflection. The native dependent theory supplies the contextual typing discipline which says when those manipulations are safe. Finally, NIK is the admission membrane between an arbitrary proposed value and an accepted inhabitant inside the same system. The derivation flow may be displayed externally for analysis, but the intended operational organization is:

```
Prime {
  languages    : LangCode values
  translations : Morphism L L' values
  held syntax  : Data_L(A)
  logic        : typed GSLT/MeTTa-IL operations
  dynamics     : spaces, match, Need, cost, and evidence
  boundary     : NIK admission of proposed operations and values
}
CeTTa realizes these operations; it does not host a separate GSLT service.
```

This internalization has four theorem-level gates. First, an internal transport combinator must have a Prime type and its denotation must be exactly the already proved functorial action on `Data`. This semantic gate is now constructed in the selected Prime CwF: source and target languages, their structural route, and held source syntax are dependent term arguments; application computes the Data push-forward; identity and composition hold; and the same route action commutes with opening, closing, lifting, and instantiation. A concrete closed Prime term supplies a positive inhabitant, while the impossible identity-symbol map from Prime back to Zero is the negative control. Exposing this term through the final concrete grammar and elaborator remains part of the definability gate, not a reason to externalize it. The gate now also has a contextual instance: language/context pairs, their context-preserving routes, and open held syntax are dependent inputs to a second Prime semantic term. Its result is indexed by the target context, preserves the exact free-name support, and proves that an absent target name cannot appear. This supplies the open-code half of I1 without pretending that semantic inhabitation already proves syntactic expressibility. Second, the ambient GSLT-IL elaboration is a proof-relevant relation, while a declared typed profile earns a direct elaborator exactly when it is covered, determinate, and thin in the proof-history dimension. On that represented fragment, exact selection supplies a companion, its conjoint, and the binding equations, and the Prime-internal fragment must correspond fully and faithfully to this represented action. The ambient relation must not be collapsed to a function: a checked authored program has one surface command with two distinct valid elaborations, hence no global exact selector. Third, admission and its optional receipts must be Prime operations and values, with the C path proved to realize them rather than bypass them. Fourth, Prime's own language definition must be a transportable, inspectable `Data` value at a strictly higher level. A standing design test follows: a proposed meta-operation is not part of GSLT-IL until it has a Prime-internal representative of the stated type. The existing hygiene and functor laws now inhabit that representative; the reverse direction of the definability theorem and the other gates remain open.

The ambiguous case now has a request-local native theorem rather than only a warning. For one authored command, let

$$W(c) = \sum_t \mathsf{Evidence}(c, t)$$

be its complete elaboration worlds, and retain an ordered finite history $W(c)^*$. Three native faces expose, respectively, the history cardinality, the list of visible internal outcomes, and the complete outcome–evidence worlds. They form an information order, but all three leave the retained semantic history unchanged. A face enters an exact NIK request precisely when every requested dependent policy factors through its readout. Therefore a complete-history request excludes the first two faces and has the full-world face as its unique strongest member. Current activation returns the entire ordered history without replaying selection; a dependency change disables that face and returns the same history to fallback.

The non-collapse theorem is the important part: this unique strongest *observer* exists even for an authored profile with two distinct proof histories and no functional elaboration representation. NIK has selected how much of an already-retained relational world a declared consumer may observe; it has not selected one world, manufactured a compiler, or converted loose semantics into a tight route. Thus natural ambiguity and maximal-native realization coexist. A language may remain relational at its semantic waist while earning direct, check-free kernels for exact observation or execution requests over that relation.

Transport between languages sharpens the boundary. An operational route does not automatically say how an elaboration proof in its source becomes an elaboration proof in its target.

Complete-world transport is therefore an additional typed-route capability with two components:

$$f_0 : \mathsf{Internal}_L \longrightarrow \mathsf{Internal}_{L'}, \qquad f_1 : \mathsf{Evidence}_L(c,t) \longrightarrow \mathsf{Evidence}_{L'}(c', f_0(t)).$$

It maps the dependent sum $W_L(c)$ and hence every ordered finite history pointwise. Identity and composition hold on both complete worlds and their lists; list length is invariant, and projecting visible target outcomes is exactly pointwise application of f_0. This is a constructed forward map, not an assertion that every loose, represented, or operational route carries proof-history transport.

Target observation policies pull back along this complete-world map. All three native faces execute the same forward semantic operation, while their keys retain different amounts of the mapped target history. The request for complete *target* worlds therefore again has the full-world face as its unique strongest realization. Revision-current NIK activation applies the retained map directly as both semantic operation and target policy input; a dependency change disables it and leaves the original source history intact for fallback. No inverse is needed for this safe forward behavior.

Reflection is a separate license. Injectivity of the map on dependent worlds lifts to injectivity on ordered histories. But a checked counterexample maps two distinct evidence constructors for one internal outcome to one target constructor. The induced list map preserves two occurrence positions, yet the two singleton source histories have the same mapped target history. Even the strongest complete-target-world observer therefore cannot distinguish them, while stale fallback still can. In implementation terms, "complete" is always relative to the retained fibre: NIK may advertise source-history replay or source-sensitive caching only when the route additionally carries the reflection proof.

The transport capability is now attached to the intrinsic typed route syntax, rather than floating beside it as an arbitrary evidence function. Over a validated presentation, an evidence profile consists of an authored elaboration program and its proof-relevant command fibres. A typed route between two such profiles contains a structural presentation morphism, a map of authored commands, an executable map of internal patterns, a proof that the executable map is pointwise the structural symbol action, a surface-naturality law, and the dependent evidence action above. Identity and composition act on all of these fields together. Assigning this displayed route data to every primitive generator therefore extends over the freely composed intrinsic route programs, and forgetting the displayed fields yields exactly the pre-existing structural path interpretation.

This supplies the missing constructional input to maximal-native selection. Restricting a typed route at one authored command produces the complete-world map consumed by NIK, so current execution runs a function carried by the language operation itself; no route-name dispatcher or unrelated runtime transport is introduced. Generator-level reflection proofs compose to injectivity for every intrinsic route program. Conversely, the current typed `promote` program admits both a rich displayed interpretation which retains two distinct proof histories and a forward interpretation which maps them to one target history. Both lie over the identical structural morphism. Thus structural route typing determines safe forward transport, while exact proof-history reflection is an independent capability. The strongest kernel is consequently selected relative to a declared request and proved capability set, not guessed globally from the route's spelling.

That last boundary is now exact for arbitrary dependent policy families. Let h map an ordered source proof history to its transported target history, and let v_P be the complete vector of answers requested by a policy family P. The route supports P precisely when there is a fixed runner r with

$$v_P = r \circ h.$$

Equivalently, every pair of source histories identified by h must agree under every policy in P. The factorization carries executable runners and installs directly into NIK's ordinary policy catalog; the

fibrewise equivalence is a specification theorem, not a runtime synthesis oracle. Global injectivity is sufficient for every P, but is not required for a particular request, and composition cannot recover a policy distinction already lost by a prefix. In the checked canary, the non-injective `promote` route exactly preserves history length and is therefore admissible for that policy, while the same route is rejected for complete source-history replay. This is the operational meaning of "strongest justified kernel": maximality is taken inside the exact request fibre, not in a universal order that would discard useful lossy realizations or silently erase provenance.

Policy adequacy also composes, but its exact law is image-sensitive. Suppose the first route maps histories by h_1 and realizes the source policy vector as $v_P = r_1 \circ h_1$. The function r_1 is the residual policy family on intermediate histories. If the second route h_2 carries runners r_2 with $r_1 = r_2 \circ h_2$, executable composition gives

$$v_P = r_2 \circ h_2 \circ h_1.$$

This is the constructive forward rule used by a composed native realization.

The converse is true without qualification only on the propositionally certified image of h_1. A reached intermediate history carries the proposition that some source history maps to it. Composite policy support is equivalent to support of the residual family after restricting h_2 to precisely those reached histories. If h_1 is surjective, this becomes an equivalence over the full intermediate history type. Without surjectivity the unrestricted statement is false: a prefix runner may distinguish an unreachable intermediate history, the suffix may erase that distinction, and the source-to-target composite may still answer every source policy exactly. The formal counterexample proves all three facts at once—composite adequacy, ambient suffix refusal, and non-surjectivity—while the reached-image suffix remains admitted. Thus NIK neither rejects a safe composition because of impossible intermediate worlds nor advertises authority over worlds the composed program never reaches.

This image certificate must not be confused with retained provenance. The image subtype stores an intermediate history together with a proposition that some source reached it; it need not retain that source history as runtime data. The stronger `ProvenancedReachedHistory` stores the source and target histories plus their route equation. It is equivalent to the complete source history, and its forgetful map to the propositional image is surjective. That map is injective exactly when the typed route reflects complete histories. Consequently identity supports a direct provenance-replay policy, whereas the same collapsing route which safely supports history length provably refuses source-provenance replay: two distinct source proof histories have the same reached target key. Exact replay, tamper checking, and provenance-sensitive caches must therefore retain the stronger key.

The provenance carrier is constructional rather than archival. A suffix maps it forward while retaining the original source endpoint, and the two parenthesizations of a three-route chain are canonically equivalent. The resulting transport is associative under that equivalence. Thus a native kernel chain may use the smaller reached image for a policy request and the larger provenance carrier for exact accountability, without either changing the route semantics or making cache identity depend on compiler parentheses.

This architecture changes ergonomics because effects formerly hidden in a broad type become visible, typed term formers. The target operator boundary is:

Syntax	**Native target meaning**	**Implementation status and consequence**

`Atom`	inert top of the ordinary value order	The current authored Prime language still uses broad `Atom` fields. Removing evaluation control from this type is a migration target, not a completed source change.
`@`/quotation	introduction of language-indexed held syntax $\mathsf{Data}_L(A)$	Current Prime already has structural quotation. The type index, contextual hygiene, and cross-language action are proved in the separate Data layer, not yet one serialized production operator.
`!`	counit/evaluation of an already admitted Data value	The Lean operation is proved check-free after admission. The present langdef spells the crossing as `prime-evaluate-name`; adopting `!` as uniform syntactic sugar still requires elaborator and runtime work.
`Need`	demand scheduling under a revisioned authority	It remains a runtime control operator. It does not become proof checking and does not require an interior certificate trail.
space polarity	evaluate-by-default or data-by-default elaboration mode	This controls where quotation/evaluation syntax is inserted; it does not identify `Data`, gradual unknown, or top, and it does not alter the kernel judgment.

Thus the core mathematical operators do change at the *native target*: holding and running become a typed pair, and language translation becomes an operation on held syntax. The live Prime langdef has not yet been destructively rewritten to those spellings. Compatibility profiles can preserve HE/PeTTa calling conventions while a typed translator inserts the explicit operations. The important ergonomic invariant is that a user pays syntax at a boundary, not checks on every interior reduction.

Lean4Less supplies an independent external calibration of the equality-profile architecture. It translates selected ordinary Lean terms to a smaller Lean-minus profile by replacing definitional proof irrelevance and K-like reduction with explicit transports, then checks the result with a specialized Lean4Lean kernel. This follows the extensional-to-intensional translation line of Winterhalter et al. The bounded NIK integration skeleton accordingly treats ordinary Lean and Lean-minus as two native proof-system guests with exact proof- fibre parity, connected by a judgment-preserving profile translation and a noncollapsing universe-order embedding. It does not claim that the external translator has discharged these laws for all Lean: the audited implementation still reports scaling limits, possible transport explosion, deliberately aborted declarations, and a currently broken `.olean` output route. Lean4Less is therefore evidence that explicit profile transport is a real architecture, not a completed Lean-as-NIK instance.

7.5 Pure is an exact source fragment, not the owner of the native theory

There are two distinct Pure developments. The intrinsic `PureKernel.HasType` uses de Bruijn terms, declarations, universes, Π/Σ/Id, conversion, and inductive infrastructure. The Pattern presentation uses locally nameless terms and the extrinsic judgment `PureHasType`. The native-theory development now proves a substantial relationship for the intrinsic side:

```
intrinsic PureKernel
    -- faithful, exact on its image; substitution and typing commute -->
native raw terms and the native contextual support
```

The embedding has projection, round-trip, image, and injectivity theorems. Renaming and substitution commute with it, typing is exact on its image, and the typed context/substitution map is faithful. It is not full: staged substitution and quotation provide named native terms with no typed preimage in intrinsic Pure. Thus the native theory conservatively retains the Pure fragment while strictly extending it with Pattern collections, staging, costs, evidence, and language codes.

The other side of the triangle is now sharply delimited, but not yet closed. The two authored typing judgments are not merely notational variants. `PureKernel.HasType` omits domain or codomain well-formedness premises from several introduction and elimination rules which `PureHasType` records explicitly. For example, the raw intrinsic relation types a lambda whose domain is the untyped top universe; a new regular intrinsic spine rejects that derivation and forgets soundly into the raw relation. Conversion in the regular spine is intensional, but conversion to an ordinary target now requires a derivation that the target is a type; conversion to the untyped top sort is a distinct rule. Raw telescopes are likewise separated from regular contexts: the raw syntax admits a top-sort assumption and variable typing consumes it, while the regular context formation judgment rejects the same telescope.

This selection is no longer merely editorial. The regular rules are packaged as a rule-model interface, and the inductive regular judgment maps into every relation closed under that interface. It is therefore the *least* rule-closed relation generated by the stated finite rules. The inclusion into raw `PureKernel.HasType` is proved, and three independent ablations prove that the reverse inclusion fails: raw typing accepts a lambda with an unformed codomain, reflexivity over an unformed carrier, and a dependent pair whose declared domain is the untyped top universe. The regular rules reject all three. Removing any corresponding premise therefore strictly enlarges the calculus.

The structural metatheory now proves why these local repairs compose. Regular conversion commutes with renaming and declaration-free simultaneous substitution; regular typing is stable under renaming, weakening, substitution, and singleton instantiation; and formation of the components of syntactic Π, Σ, and identity codes is invertible. Consequently, in every regular context, each typing result is either the distinguished untyped sort `U1` or is itself typed by `U1`. Presupposition closure is a global theorem of the judgment, not a review convention attached to selected rules.

Two tempting categorical slogans are deliberately withheld. An inductive typing calculus is a least fixed point of finite derivations, not a greatest fixed point; the latter would admit cyclic or infinite evidence. And "coreflection" is earned only after a category of presentations and the required adjunction are actually defined. The present theorem is the precise initial/least-closure property in the preorder of typing relations, not an unproved adjunction.

The prospective native kernel should therefore enforce context and type regularity by construction, with `PureKernel.HasType` retained as a permissive raw presentation rather than silently crowned as the final MeTTa judgment.

The syntax boundary has also been made exact. Declaration-free intrinsic terms quote into

PureTmPattern, and a tagged constant quotation has Pattern-purity if and only if its source is declaration-free. A negative constant witness is proved. The exercise additionally exposed a real defect in the legacy artifact quote: it renders a declaration name directly as an application label, so a declaration printed as U0 collides with the reserved universe constructor. The tagged quote removes that collision while agreeing byte-for-byte with the legacy quote on the constant-free fragment. The exact boundary no longer uses the declaration-name pretty-printer at all: names are encoded structurally by their anonymous, string-component, and numeric-component constructors. This encoding is proved injective. The whole exact term quote is consequently proved injective under an injective, binder-compatible variable environment; the binder case reopens the two locally closed bodies and uses the close/open cancellation law. Regular context formation implies that every telescope entry is in the common syntax, and equality of quoted regular contexts now reflects to equality of intrinsic telescopes. Thus syntax and contexts cannot hide a mismatch. What remains is the full intrinsic–extrinsic typing equivalence for the regular conversion-bearing spine.

The forward half is now proved. A substitution-derived change-of-opening-name theorem transports both conversion and typing across the cofinite binder names, without postulating alpha equivalence. Every declaration-free intrinsic reduction constructor maps to Pattern conversion; an explicitly fragment-internal conversion proof maps recursively; and every regular intrinsic typing derivation, including dependent formation and elimination, maps to PureHasType. The identity function supplies a positive end-to-end witness. The conversion proof fibre needed strengthening during this proof: requiring only constant-free endpoints was insufficient, because raw symmetric conversion can leave the fragment through a term containing a declaration constant and later return to a pure endpoint. The replacement relation certifies every reduction edge, reflexive endpoint, and intermediate term as declaration-free. A concrete constant-bearing beta detour proves that this restriction has content.

Unrestricted reflection into the regular intrinsic spine is now proved false, even when all displayed terms lie in the exact constant-free image. The authored Pattern conversion rule can assign U0 a beta-redex type convertible to U1 without requiring that redex itself to be a well-formed type; the regular intrinsic judgment rejects exactly that derivation. Therefore the reverse square must use a Pattern boundary whose presuppositions are intrinsic, or an elaboration into the regular intrinsic kernel. It may not be obtained by silently identifying the two authored judgments. Reflection for that corrected boundary remains open. "Pure" should consequently name an admissible region of related intrinsic and presented authorities, not whichever encoding was implemented first.

The sound computational conversion component of that corrected boundary is now available. One-step and finite reduction are proved to preserve the declaration-free fragment; complete development therefore yields a fragment-internal conversion proof. The executable regular comparison uses the same complete-development computation as the earlier checker, but returns the strengthened conversion relation whose every edge, endpoint, and intermediate term remains in the common fragment. Positive universe reflexivity and negative cross-universe computations prevent the comparison from being vacuous.

The old one-pass comparator is not a complete decision procedure. A declaration-free two-step redex is proved fragment-internally convertible to a variable while the current one-pass development comparator returns failure; this lifts the older raw incompleteness observation into the regular proof fibre. A production regular checker must therefore use a genuinely complete normalizer before it can supply a DecidedRelation. The executable part of that replacement now exists: an outermost-leftmost selector carries a proof of every chosen reduction, failure is equivalent to irreducibility, and accessibility of the reduction relation drives normalization to a unique normal form. On the explicit domain "reduction-accessible and declaration-free", this induces an exact certificate-free DecidedRelation; a beta-redex reducing to U0 is the positive computation and

`U0/U1` is the negative computation. The self-reducing declaration-free omega term is proved outside the domain, so syntax purity cannot masquerade as termination. The required logical-relations theorem is now proved: every subject admitted by a full regular judgment is reduction-accessible. This promotes the existing normalizer from an explicitly supplied accessibility domain to every admitted regular subject. Building a production bidirectional checker which constructs the full regular judgment remains a separate realization task. The first reducibility layer for that proof is now formal. Accessibility is closed under dependent function, dependent pair, identity, lambda, pair, and reflexivity constructors, and it reflects from applications and projections to their components. Scoped reducibility candidates carry CR1–CR3, admit intersection, nondependent function-space, and product-space constructions, and have concrete positive and negative inhabitants. In particular, the identity lambda inhabits the normalizing function candidate while the delta self-application does not. A separate theorem proves why this layer cannot be replaced by structural induction: two individually accessible delta components form the non-accessible omega application. Reduction steps now also reflect through arbitrary variable renamings, so accessibility forms a renaming-stable family of candidates across de Bruijn scopes; injectivity of the renaming is not assumed. This is exactly a presheaf-level result, not yet a Kripke model. A scope-aware function candidate now quantifies over every future renaming and every argument admitted in that future scope. Its candidate laws are proved using a fresh-variable probe, reflection of reduction through renaming, and accessibility induction on arguments. The construction includes positive identity and negative looping-delta witnesses.

The first argument-indexed extension is also formal. A dependent codomain selects a reduction candidate from the actual argument and supplies the exact backward conversion transport needed when that argument reduces. Constant codomains recover the nondependent function candidate definitionally. A second saturated candidate—strongly normalizing terms which never reduce to `U0`—then supplies a genuinely nonconstant codomain: `U0` belongs to the fibre indexed by `U0` but not to the fibre indexed by `U1`. Thus argument dependency is witnessed rather than represented by a constant-family placeholder. At this intermediate layer, substitution naturality and the interpretation of syntax were not yet available; the subsequent semantic-universe construction supplies them.

The need for that stronger layer is forced by a separate negative theorem: the open self-application is accessible, and the substitution sending its variable to the accessible delta term is pointwise normalizing, yet the instantiated term is omega and is not accessible. The proof object must therefore be a genuinely dependent, type-indexed candidate semantics for contexts and candidate-respecting substitution; this identifies the fundamental-lemma boundary rather than an unfilled evaluator case. The constructor-specific head-expansion interface needed at that boundary is now formal as well. Exact beta expansion requires accessibility of the lambda body, the argument, and the instantiated contractum; first- and second-projection expansion require accessibility of both pair components. These laws are stable under semantic weakening and reindexing, and are required only over environments realizing their contexts. They cannot be replaced by generic backward closure: an erasing beta redex with the looping omega term as its argument reduces to `U0`, although the redex itself is not accessible. Thus the fundamental lemma must consume the exact introduction laws, not assume that reducibility is closed under arbitrary reverse reduction. A realized-context refinement now states the candidate laws only for substitutions which realize their semantic context. Each such environment still yields an ordinary reduction candidate with the exact head-expansion interface, while context extension no longer demands semantic candidate data for malformed tails. The earlier coherent candidates embed conservatively: their normalizing predicates agree pointwise. Contextual extension, weakening, reindexing, head-variable realization, and positive/negative normalization canaries are proved. This is the interface needed for a genuinely

dependent function candidate whose closure proof may consume context realization. That candidate is now constructed: its codomain is selected in the context extended by the actual argument, and argument reduction uses both directions of environment coherence. CR1–CR3, renaming, base-environment reduction, and exact beta and projection expansion are proved. The identity lambda is a positive inhabitant; the accessible delta function is rejected because its self-application is omega. The later semantic-universe layer interprets the syntactic type formers with this candidate, and the simultaneous fundamental lemma below consumes these exact laws. The dependent pair candidate is now constructed at the same realized-context boundary. A pair is observed through both projections: the first projection is checked in the base environment, while the second is checked in the environment extended by that actual first projection. Pair reduction therefore moves both the observed second component and its codomain environment. CR1–CR3, renaming, both directions of environment-reduction coherence, and exact beta and pair-projection expansion are proved. Pair introduction explicitly transports the second component from the environment indexed by the written first component to the environment indexed by the pair's first projection. A pair of `U0` terms is the positive witness; a pair with omega as its second component is rejected.

Universe, dependent-function, dependent-pair, and identity codes now have a proof-relevant semantic-universe relation retaining their exact contextual candidate meanings. Semantic contexts retain the candidate selected for every telescope entry. Candidate-respecting substitutions lift through dependent extensions, support comprehension and singleton instantiation, and commute extensionally with both dependent-function and dependent-pair construction. Semantic type codes pull back along every such substitution: the transported meaning is extensionally the reindexing of the source meaning. This is the substitution theorem required by language-internal transport, not an external checker correspondence. Conversion is also coherent with the retained proof-relevant meanings: convertible semantic type codes select extensionally equivalent contextual candidates. The dependent-function and dependent-pair cases use Church–Rosser constructor injectivity and transport the dependent codomain across the semantic identity substitution induced by equivalent domain meanings. Consequently, a fixed semantic type code has a meaning unique up to the observable extensional equivalence, without erasing its dependency structure or identifying proof records definitionally.

The conversion guard needed by the simultaneous proof was strengthened during this construction. Raw fragment conversion does not transport normalization backward: an erasing beta redex containing omega converts to `U0` while remaining non-normalizing. Merely requiring accessibility at the identity environment is also too weak for dependent substitution. Semantic conversion therefore requires both endpoints to normalize after *every* environment realizing the current semantic context. The guard is symmetric, transitive, and stable under semantic substitution. A beta-hidden dependent function code is the positive witness; looping and erasing-omega codes are negative controls. The target-formation premise of regular conversion is consequently load-bearing evidence for the logical relation, not redundant syntax. The simultaneous formation-and-membership fundamental lemma over this universe is now proved by induction on the regular typing derivation. It interprets formation and ordinary membership together, extends semantic contexts at dependent premises, and uses conversion coherence to transport terms without identifying proof records. During this proof, application and both dependent-pair projections exposed a further presupposition gap: their domain formation had previously been recovered from another premise. The regular constructors now require domain formation explicitly, so the proof is structurally recursive and the kernel judgment carries every premise its semantic interpretation consumes. Every regular context consequently has a semantic realization; every full regular judgment has an accessible subject. The identity program is the positive witness, while the self-applicative omega term is proved to have no regular judgment at any type. Identity witnesses require no stronger candidate in the present frag-

ment, because identity has reflexivity but no eliminator. The normalization interpretation therefore uses the contextual normalizing candidate and proves its introduction rule from accessibility of the reflected payload. This is explicitly not a semantic identification of endpoints. Reflexivity of `U0` is the positive witness; reflexivity carrying omega is rejected, so the interpretation does not hide divergence. The older algorithmic checker also returns the permissive authored typing relation, so it is not renamed or promoted: the new bidirectional checker must return the regular judgment and discharge formation premises on every fast path.

OSLF and LADL guide the next promotion step, but do not choose an authored typing judgment by themselves. OSLF supplies operational space and behavioral models in which each candidate premise can be tested; a rule is promoted only after its interpretation is sound and nondegenerate. Behavioral equivalence remains an explicit modal proposition or equality profile and never leaks into kernel conversion. LADL can derive collection-sensitive connectives only after the relevant term/collection distributive law is constructed. In the present theory, bags retain occurrence-relevant evidence while set quotients retain theoremhood; the distributive law connecting that distinction to the complete native syntax is still a named construction target, not an automatic consequence of the GSLT.

7.6 Four native modes of checking and reasoning

The NIK waist does not reduce every guest to transcript replay. It supports four modes, each with a different evidence boundary.

Mode	Native action	Evidence and cost boundary
Direct decision	A `DecisionKernel` computes a semantic judgment directly.	The induced checker has unit evidence; ordinary typing and decidable conversion need no certificate language.
Native proof kernel	The kernel computes the claim proved by the guest's own proof object.	Exact proof-fibre equivalence identifies accepted certificates with native proof objects, rather than merely decoding one direction.
Admitted operation flow	Operations proved once to preserve meaning and acceptedness compose accepted premises into accepted conclusions.	No observer or checker runs in the interior path; rechecking is needed only when an untrusted value crosses the boundary.
Certificate boundary	An untrusted searcher or importer supplies the finite choices which deterministic kernel computation cannot recover.	The kernel reconstructs consequences and checks once; receipts and exported proof artifacts are optional boundary products, not permanent runtime luggage.

The exact-fibre requirement is essential. A checker may accept a proof object decorated with an ignored tag and still have a sound decoder, while carrying more accepted evidence than the native proof system. The formal ignored-tag counterexample rules out calling such a format canonical parity. Compressed or multi-encoding formats must instead provide an explicit normalization quotient.

direct decision	**native proof**	**admitted flow**	**boundary evidence**
$J(c)$ computed	$p : c$ computed	$f(p_1, \ldots, p_n)$	choices supplied
unit evidence	exact proof fibre	sound once, flow freely	reconstruct once
no replay	proof objects retained	no interior recheck	receipt/export optional
One logic-polymorphic waist; the guest selects the mode. The modes may coexist within one authority at different trust boundaries.			

Figure 1: The four NIK execution/proof modes. Certificate carrying is one boundary mode, not the ontology of NIK or the default typing path.

The converse boundary is also formal. A computable step-budget checker gives sound and complete finite evidence for fixed-input halting, while no computable `DecisionKernel` decides the corresponding predicate. Certificates can therefore add genuine power at a search boundary, but they add nothing to a judgment already decided by a native kernel. Megalodon instantiates the middle of this spectrum concretely: its native proof kernel computes the result of a Mathdata proof object, has exact canonical proof-fibre parity, and decides the explicit fuel-bounded $\beta/\eta/\delta$ conversion relation. The fuel bound is part of the stated relation; this is not an unqualified normalization theorem.

Contexts and consequence are optional doctrines above this waist. Cartesian weakening and contraction are never forced on a resource-sensitive guest; named counterexamples prove that a resource context can admit neither duplication nor discard. Proof erasure into a thin consequence relation and hence a π-institution-like view is likewise an optional reflection, not the internal logic of raw NIK and not evidence that GSLT-IL is the internal logic of Ind(ProofGSLT).

7.7 The Maximal-Native-Calculus Principle

The policy name is the **Maximal-Native-Calculus Principle**; its precise formal content is *structure-licensed maximal-element selection*:

> For each guest, semantic request, certified fragment, and current dependency revision, recognize a finite family of admitted calculus realizations, order it by a declared license order, and select a maximal licensed member. Do not flatten a guest to generic replay, and do not impose structural rules or completeness claims which its presentation does not earn.

This is now packaged as a theorem. A `RecognizedFamily` contains a finite recognized set, a nonempty licensed subset, and an admitted operation at every index. The declared order must be capability-monotone, and every strict comparison must exhibit a capability gained by the stronger realization, so an arbitrary index order cannot masquerade as calculus strength. Finiteness yields a maximal licensed index; selecting it adds no checker or replay layer, and semantic preservation follows from the selected admission arrow. "Maximal" does not mean "greatest": a proved two-element discrete counterexample has two incomparable maximal choices and no greatest choice. The exact positive boundary is also proved. If the licensed family is upward-directed—every two licensed realizations have a licensed common upgrade—then every maximal selection is greatest and therefore dominates all licensed alternatives; partial-order antisymmetry makes that greatest

member unique. Thus a guest may advertise a canonical strongest kernel only after supplying common-upgrade closure; otherwise NIK exposes the maximal antichain and an explicit observation or cost policy selects among its incomparable members. Nor does it mean globally fastest: the license order concerns certified capability, while cost is a separate verified refinement.

Selection is now formalized at the granularity at which a runtime can honestly make that claim. A `CapabilityRequest` cuts out a finite fibre by an exact if-and-only-if law: its candidates are precisely the currently licensed realizations supporting every requested capability. This prevents a dispatcher from manufacturing a "strongest" answer by simply omitting a stronger candidate. The operations and their semantic-preservation proofs are unchanged by restriction. If the *request fibre* is upward-directed, it has a unique greatest member, and the selected admitted operation still runs directly. It is not necessary that the whole guest-wide family be directed: direct decision, proof construction, admitted flow, and boundary replay may be globally incomparable while a particular construction or checking request has one exact strongest realization. Conversely, a checked request with two incomparable eligible realizations has no canonical strongest kernel; the formal negative example rules out replacing that fact by a hidden priority order.

Observation requirements enter this same exact fibre rather than forming a second dispatcher. A displayed policy catalog assigns each native realization a readout, a proof-relevant runner for every supported policy, and an agreement law with the policy's semantic answer. The augmented capability universe is the disjoint sum of native capabilities and policy indices; candidate membership is the conjunction of the original native request and support for every policy in the requested dependent family. Hence the existing maximal, greatest, and profitability-frontier selectors apply unchanged, while every selected member retains an executable realization of the complete policy family and factors onto its least sufficient answer vector. The negative bounds are sharp: mutually unsupported policies yield an empty fibre rather than an invented runner, while an empty policy request leaves incomparable native calculi incomparable rather than manufacturing a greatest one. Computational strength and observational adequacy are therefore independent request coordinates which meet only in an exact eligible fibre.

Retention is likewise shared rather than synchronized by convention. A generic revision-indexed policy admission stores the executable family realization at one NIK dependency revision; its prepared state contains both the selected readout and the complete semantic state. A semantically maximal product candidate then retains its original native operation and this policy admission at the same revision. Maximal, genuinely greatest, and cost-selected frontier members all construct the same retained object. One currentness witness activates both hot functions, while a relevant dependency change makes both activations unconstructible and leaves the untransformed operation input and complete policy state available for fallback. The generic policy layer is universe-polymorphic; the present adapter for executable admission objects is intentionally runtime-sized, pending the separate higher-universe operational generalization already identified by the native-admission boundary.

Policy-family support is contravariantly stable under arbitrary semantic state maps. If $m : X \to Y$, a family F on Y is executable from readout r, then m^*F is executable from $r \circ m$ using the identical retained runners. Support reflects back to Y when m is surjective. This boundary is exact: a proved non-surjective counterexample restricts away the target states which distinguish a policy, so its source readout becomes sufficient although the target readout is not. Consequently route transport preserves every already justified native realization, but may reveal additional image-specific realizations only through a fresh recognition theorem; transport itself never mints them.

The same law now transports maximal-native selection. Pulling back a policy catalog leaves its native realization family, capability order, declared support, and exact request fibre unchanged, while precomposing only its states and readouts. Therefore a maximal selection, or a genuinely

strongest one when the request fibre has common upgrades, remains such in the transported catalog and runs the same retained function under the same revision witness. This is maximality relative to the exact transported catalog, not a claim that a non-surjective route has already enumerated every stronger kernel newly valid on its image. The distinction permits aggressive language-relative native kernels without confusing a safe transported baseline with global optimality.

The resulting dispatch law is therefore

$$(\text{guest}, \text{judgment}, \text{fragment}, \text{revision}, \text{required native capabilities}, \text{required policy family}) \longmapsto \begin{cases} \text{the unique greatest licensed realization}, & \text{if the exact fibre has common upgrades}, \\ \text{the maximal frontier plus an explicit policy}, & \text{otherwise}. \end{cases}$$

Only realizations with the same semantic source and target are compared inside one fibre. A native constructor, a proof-producing kernel, and a certificate replayer with different boundary contracts are related by proved adapters or composition, not by declaring one globally larger. Compatible kernels may earn a common upgrade—analogous to a reduced product of abstract interpretations or a cooperating combination of decision procedures—but the compatibility and preservation square are obligations, not dispatcher folklore.

The first nondegenerate hosted instance compares a guest-native proof kernel with the same kernel exposed at an explicit certificate boundary. They accept exactly the same native proof fibre, but their retained route receipts differ. A request for exact proof identity and certificate-free use selects the native face uniquely; a request for exact proof identity at an explicit trust boundary selects replay uniquely. With neither boundary capability requested, both faces remain eligible and incomparable, and a checked negative theorem proves that no strongest member exists. Selection is also dependency-current: an irrelevant revision-coordinate change retains direct native execution, whereas a relevant selected-dependency change makes activation unconstructible rather than silently choosing another face. Even direct theoremhood decision and native proof checking can share the same accepted claims and the same certificate-free Boolean while remaining different entry constructors. Thus neither a fixed mode priority nor a single "needs certificate" flag implements the maximal-native principle.

The supporting theorem family places this selection between exact positive and negative bounds:

- direct semantic decision embeds as a full authority with no proof details;
- exact native-proof parity prevents certificate inflation;
- admitted proof operations compose and preserve acceptedness with no interior observer;
- resource guests refute an ambient cartesian default;
- the halting boundary refutes universal direct decision;
- general reachability analysis, dead-code elimination, nontrivial property-driven optimization, and universal optimal compilation meet explicit impossibility theorems; and
- every explicitly finite fragment has a correct optimal compilation witness.

These results yield a three-band engineering discipline:

A future calculus license should therefore be an admitted, revisioned claim about a guest or fragment: it names the required structural capabilities, the executable engine, its soundness and

Decidable band	Compute the judgment directly.	No certificate language; use the admitted fast path after checking.
Searchable band	Retain only genuine choices made by search.	Reconstruct deterministic consequences and check once at the boundary.
Open semantic band	Stratify theories and authorities.	Do not promise a finite complete calculus, same-level self-certification, or a universal optimizer.

compute $\longrightarrow$ certify choices $\longrightarrow$ stratify as decidability decreases.

Figure 2: The three-band discipline. Finite evidence reaches the recursively enumerable/searchable band, not arbitrary semantic truth.

adequacy scope, and any completeness or cost result. It should not be one giant options record and should not enlarge the public NIK checking ABI. This is also the common algebra for proof-rule admission, verified MAM transformations, and Prime runtime specialization: a transformation proved once to preserve the relevant invariant may thereafter run on the fast path.

7.8 Accelerate, do not guard: checking is ingress

The native dependent theory is not an ordinary evaluator surrounded by a type filter. Raw relational computation remains meaningful without a native typing derivation. Conversely, once an exact typed object has been constructed, its introduction, elimination, transport, and admitted compositions construct their result judgments directly. Re-running a checker in those interiors would discard the principal benefit of intrinsic typing. The intended dependency is therefore

```
raw Pattern
  -- exact decode --> indexed query
  -- check once ---> intrinsic evidence
  -- construct ----> typed values and native operations
  -- admit --------> a current maximal native realization
```

The current declaration-aware fragment now realizes this distinction at a nontrivial boundary. A telescope of arity n carries a `LevelSpine n`: one explicit universe annotation for each declaration. The intrinsic judgment `StructuralContextFormation` has an empty constructor and a snoc constructor whose premises retain both the previously formed telescope and a derivation $\Gamma \vdash A : U_\ell$. Ordinary typing and context formation have disjoint canonical codecs. Their coproduct is therefore one exact heterogeneous judgment waist, and `primeMeaning_equiv_sumFibre` identifies its universal proof fibre with the product of the two independent semantic views.

Context formation is a validated `newJudgmentsOnly` extension of the structural presentation. The forward theorem compiles every intrinsic context tree to an accepted raw artifact. The reverse theorem `exists_accepted_raw_iff_nonempty_contextFormation` reconstructs the intrinsic telescope from every accepted artifact at a canonical query. This is not tautological checker parity: the checker consumes rule schemas and raw trees, whereas the interpretation targets an independently defined indexed family and retains its ordered premise evidence.

Formed typing now extends that same rooted presentation rather than accepting a raw telescope on the side. Its query retains the context level spine and the result universe level; its derived rule has three ordered premises:

$$\mathsf{Formed}(\Gamma, \vec{\ell}), \qquad \Gamma \vdash A : U_\ell, \qquad \Gamma \vdash t : A.$$

The generic checker accepts the derived node exactly when the three component artifacts are accepted, and accepted packages reflect into `IntrinsicFormedTyping`. The syntactic contextual object is then built from the reflected context evidence; no checker is stored in the resulting native term. Swapping the two typing children is rejected, a wrong result-universe level is uninhabited, and an undeclared constant in the telescope supplies the decisive negative control: both old typing components remain inhabited, while the complete formed judgment and every accepted formed package are impossible.

Authored nominal declarations now cross the same boundary without replacing source order by an extensional environment. An executable `activeInventory` traverses the complete declaration list and retains a proof of the exact first occurrence for each active constant. It emits a finite table of ground inference facts with collision-free identifiers; the ordinary V2 extension validator admits that table only as a fresh judgment. The theorem `checkRaw_reflects_active_source` then classifies every accepted fact application and reconstructs its source occurrence, source index, entry, and displayed type. A concrete active declaration has an exact one-node raw proof. In the paired negative, every possible raw proof tree for a later same-name declaration is rejected. Thus generated checking is an honest ingress for authored data, while already constructed hosted judgments continue through the stronger native route without interior replay.

Authored equations now cross an analogous boundary without being flattened to endpoint equality. A proof-relevant path through the complete declaration document records both the source position and the position in the filtered equation inventory. Consequently constants may be interleaved with equations and two equations with identical endpoints remain distinct occurrences. The generated ground-fact presentation passes the same V2 extension gate. Accepted artifacts reflect to the exact path, while a claim whose source index points to an interleaved constant is rejected for every raw proof tree.

This boundary also exposes why the generic checker is a floor rather than the maximal native realization. The exact codec represents a de Bruijn variable as closed codePattern data. Generic schema instantiation therefore leaves that encoding unchanged, whereas Prime simultaneous substitution replaces the object-level variable. The two operations are formally unequal on a concrete open term. An authenticated equation enters native computation only after Prime substitution and endpoint typing; if the hosted declaration world carries uniform preservation, source typing constructs target typing without another check. The resulting proof-relevant native receipt decodes to the exact source occurrence and substitution. Thus a boundary proof says "this authored rule is present," while the native DTT says "this typed instance computes."

The distinction has both positive and negative native witnesses. The canonical List eliminator equation is constructed from its exact authored occurrence, identity substitution, and intrinsic endpoint derivations, after which its conversion runs with no checker argument. Conversely, the same authenticated List schema has a raw substitution instance containing an undeclared element; that raw reduction exists, but no typed occurrence at any proposed result type can be constructed. Source authenticity, raw dynamics, and typed computational authority are therefore three related but non-collapsed capabilities.

This negative control explains "accelerate, do not guard." It does not revoke the raw term or its ordinary relational execution. It prevents that term from claiming the stronger native capability whose construction requires a formed world. Exact evidence may authorize typed dispatch, representation changes, proof-program composition, or deletion of impossible defensive branches; absence, staleness, or conflict leaves the raw semantics available. NIK hosts this distinction and

selects the strongest licensed native realization. It does not turn typing failure into object-language falsehood, nor does it reduce Prime to repeated proof replay.

The higher-order relational fragment supplies a concrete capability ladder. An intrinsically typed hypothesis program denotes a proof-relevant relation. If each primitive relation has an exact finite evidence fibre, structural recursion derives an exact finite provider for the whole program. The chain case composes providers by dependent sum: an answer index retains the first occurrence, its actual intermediate target, and a second occurrence in the fibre over precisely that target. Hence native search materializes every target–derivation occurrence without collapsing equal endpoints or losing the middle witness. A successfully checked raw proof constructs this intrinsic program and its provider once at ingress; the resulting run operation consumes only that constructed plan and returns the complete occurrence bag.

The capabilities are ordered by theorems, not a fixed mode number:

$$\text{functional representation} \Longrightarrow \text{exact finite evidence search} \Longrightarrow \text{proof-relevant relational meaning.}$$

Both converse temptations are blocked. A finite two-answer primitive earns an exact native search while admitting no functional representation, and a valid relation with a countably infinite proof fibre admits no finite provider at all. It remains meaningful in the ambient relational semantics. Thus NIK may choose direct compilation for represented relations, complete finite search for finitely enumerable nondeterminism, or relational execution for the open case; none of these choices changes the authored relation, and no weaker capability may be promoted merely because it is operationally convenient. The checked grandparent chain is the positive end-to-end witness; an ill-shared middle is the refusing control and cannot acquire a native-search plan from otherwise valid primitive providers.

Strictly-positive families preserve this capability structurally rather than through datatype-specific evaluators. For the native List relator, the empty spine has one proof occurrence and a cons spine has the product of one exact head occurrence with the recursively exact tail occurrence. The polynomial List representation transports this index to the complete native `map-rel` evidence fibre. Thus a finite provider for elements induces a complete finite provider for Lists, retaining equal-target multiplicity at every position. If the element relation is functional, the stronger representation theorem identifies this relator with the graph of native `map`; if it merely has two finite proofs, both survive; if its singleton fibre is countably infinite, no finite List provider can exist. These three cases turn "choose the strongest native kernel" into a proved capability order rather than a datatype dispatch convention.

The construction is not specific to Lists. Its generic source is an indexed container: at each output index a value consists of a shape together with one parameter value at every position of that shape. This normal form generates a proof-relevant relation lifting without a datatype case split. Two container values are related exactly when they have one common shape and their stored values are related at every position. The generated lifting preserves identity and relational composition by proof-fibre equivalence; in the composition direction it retains one complete intermediate container and both pointwise derivations. It also maps every functional representation to the graph of the container map. These are constructional powers supplied by the description itself, not facts rediscovered by a post-hoc checker.

Exact finite search is generated only from the additional theorem that every position fibre is finite. The resulting provider index chooses one exact element occurrence at every position, and is equivalent to the complete lifted target–derivation occurrence. Thus a declared lifting may separately carry finite-search preservation and functional-representation preservation; these capabilities

transport across typed carrier equivalences and recurse over the generic higher-order hypothesis forms: primitive, proof-relevant chain, and data-former lift. A whole hypothesis therefore earns exhaustive search or direct execution structurally from its constituents, rather than by matching a benchmark name.

The separation is essential. The positive reader container has countably many parameter positions. It has the same lawful compositional relator and direct map theorem as List, yet lifting a two-way finite choice through it produces an infinite occurrence fibre. It therefore cannot supply a finite-search capability. A recursive indexed-polynomial description licenses native construction, dependent elimination, and iota computation; a separately represented parameter container licenses relation lifting and mapping; and finitarity licenses exhaustive native enumeration. These implications are strict. Polynomial List has both recursive and parameter-container views, but a second indexed polynomial stores its parameter in a negative function position. Its recursive family is strictly positive by construction, yet the empty-to-unit map proves that no covariant parameter action, and hence no unary container representation, can exist. The positive reader then separates parameter covariance from finitarity. This is the capability ladder NIK may select from. The remaining source theorem must elaborate an accepted authored family to its recursive indexed polynomial and prove exact constructor, eliminator, and computation agreement; an optional parameter-container view is earned only under a separate covariance/representation theorem.

The representation boundary is itself exact rather than merely extensional. An objectwise equivalence from an authored family to a container extension transports the complete proof-relevant relator: identity witnesses move through the equivalence, and relational composition reindexes the entire intermediate object while retaining both component derivations. Functional realizations and, when finitarity is present, finite occurrence providers transport through the same boundary. The converse is now proved as well. Any alleged generic finite-search lifting of the represented family transports back to its container; applying it to binary choice injects every parameter position into a distinct target–derivation occurrence. Hence a unary container supports generic exact finite search if and only if all of its position fibres are finite. Exact representation neither manufactures nor conceals this capability. Graph agreement similarly identifies the generated direct realization with the functorial family map, whose identity and composition laws follow from the representation. Consequently polynomial List reaches NIK with both nondeterministic answer occurrences intact and selects exhaustive finite search. The positive infinitary reader instead selects direct pointwise execution for a functional relation while provably refusing general finite enumeration. The parameter-negative family cannot construct this covariant bridge at all. These cases make maximality request-local: NIK chooses the strongest operation justified for the particular judgment, rather than placing construction, mapping, and search in one artificial global order.

The recursive checked List presentation now crosses the same boundary. An accepted raw proof retains its authored endpoints and exact proof identity, constructs an intrinsically typed native `map-rel` term, and supplies an occurrence that the structural provider returns in its complete answer bag. The run operation uses only that provider. A proof missing its recursive tail cannot construct the plan, although the surrounding relation remains available to ordinary relational execution. Hence checking remains ingress, native construction remains the fast path, and capability refusal remains gradual.

This ladder now feeds the revisioned NIK membrane directly. Direct mapping and finite proof search are compared only after both inhabit one contract: a typed query must produce a complete bag of target–derivation occurrences. For a represented relation, the direct operation strictly adds functional representation while retaining exact completeness and certificate-free use, so it is the unique greatest member of that request fibre. For a genuinely nondeterministic finite relation, the singleton search family is strongest without claiming functionality. The chosen arrow embeds un-

changed into a revision-indexed operational refinement; current activation runs that arrow, whereas a relevant dependency change makes activation unavailable rather than silently selecting another face. A checked recursive List occurrence survives this route, while the infinite-fibre counterexample cannot enter it. Thus "strongest kernel" is a theorem about a common semantic request and current evidence, never a global ordering of unrelated execution modes.

The operational admission interface used by this concrete runtime bridge is presently small: its carriers live in `Type 0`. The recognized-family and capability-selection mathematics is universe-polymorphic, but lifting the operational path API itself to higher universes remains a separate obligation. This scope boundary matters for Prime's cumulative tower and is stated rather than hidden by an implicit downcast.

The native-selection and observation theorems now meet on the two checked MIL clients rather than only on finite canaries. For both the learned grandparent chain and the strictly-positive `map-rel` program, the raw proof crosses the generic checker once, constructs its intrinsic typed program, derives its exact finite provider, and enters one revision-current NIK selection. The selected operation and a family of result policies are retained together. A current run returns the complete dependent search result—including every derivation occurrence—and its occurrence-count valuation without replaying the ingress checker or the selection proof. In the four-mode Data algebra, this hot segment is exactly the certificate-free *admitted-flow* mode: the checker remains at raw ingress, while the retained preservation arrow is the subsequent execution authority. A relevant revision change disables the operation and policy runners together while retaining the exact query and complete result as fallback.

This path now satisfies the abstract implementation contract generically, not only by inspection of the two clients. A checked ingress state pairs the authored raw proof with a derivation whose erasure is exactly that proof. The target state is the displayed graph of independent semantic evidence and its calculus-specific native artifact. These become proof-relevant discrete execution objects connected by an observation-preserving cell; universe lifting allows the native evidence and artifact fibres to remain at their authored levels. At a current revision the compiled trace is definitionally the retained graph embedding. The raw codec decodes to the complete checked ingress, and the receipt codec decodes to the complete evidence/artifact graph. Thus neither checking nor evidence reconstruction occurs inside active execution.

The displayed gradual law now instantiates this abstract contract directly. Complete program plans form an exact codec for authored source programs, and promoting a typed island decodes to the same complete source whether or not the candidate evidence matches. For rho execution, one gradual evidence state is encoded as one worldwise source-trace occurrence. Current exact evidence compiles to its certified operational schedule; suspension, local blame, or a stale evidence revision compiles as the exact raw branch. Staleness of the NIK implementation admission is an independent axis: it prevents activation while the prepared source trace remains recoverable through the raw codec. In the contested-purse control, the compiled trace still contains both communication occurrences in source order and their targets remain distinct. Hence an implementation may choose the strongest realization inside a raw fibre but cannot reinterpret that choice as selection among language-level alternatives.

The boundary is sharp. Generic raw checking succeeds exactly when an exact-erasing ingress exists for that same raw tree. The learned grandparent proof inhabits the complete model and a dependency change disables activation while preserving its checked fallback; the ill-shared-middle proof has no ingress state. Conversely, whenever two semantic graph points project to one artifact, no decoder from artifacts alone can be exact on both. A compact hot artifact is therefore a valid receipt only after faithfulness is separately proved; otherwise the displayed graph is the minimal safe implementation boundary.

Policy transport is now part of this abstract implementation contract. Every admitted implementation model exposes the static proof-relevant trace map underlying its current compilation. A target policy family pulls back along that map, reuses the same executable runners and dependency revision, and on a current run returns exactly the target answer after compilation. Pullback is strictly compositional. It also preserves maximal and genuinely strongest selection in the exact transported catalog, while a stale revision disables the pulled runner and leaves the complete raw trace available for fallback.

The converse requires surjectivity and is intentionally absent in general. A checked unit-to-Boolean implementation reaches only one Boolean trace. A collapsed readout is sufficient on that source image, yet it is insufficient for a target policy which distinguishes the reached Boolean trace from the unreached one. Thus implementation transport safely inherits every admitted target policy, but a stronger kernel made possible by the implementation image must be earned by a fresh image-specific recognition theorem. Transported maximality never masquerades as global optimality.

This client also exercises the policy boundary sharply. Its policy key is the complete dependent search result, so it supports exact-result and occurrence-count requests even though it forgets which native face produced the result. Two receipts with the same result and different faces prove that the same key cannot support a face-sensitive policy. The loss is safe exactly for the declared family, not globally. Conversely, the ill-shared grandparent middle and the codemap-rel proof missing its recursive tail cannot construct a native plan at all, so neither can be promoted by NIK merely because a finite provider exists nearby. Thus typed construction, strongest justified native execution, policy-relative observation, and gradual refusal compose in one concrete path while remaining distinct mathematical capabilities.

The gradual law now has a dependent formulation rather than only a slogan about plans. Over each raw value r sits a fibre of exact native evidence $E(r)$ and three evidence states: suspended, exact, and locally refuted. Precision fills a suspension but never changes r; exact evidence and current blame are rigid. A constructional map

$$(f : E \to E') = (f_0 : r \mapsto r',\ f_1 : E(r) \to E'(f_0 r))$$

always transports positive evidence. Transporting a refutation requires the additional, separately named capability $E'(f_0 r) \to E(r)$. Without that exact-reflection map, the only sound forward image of cached blame is suspension. Formed-context substitution instantiates the positive law using the native substitution theorem; revision mismatch instantiates invalidation. Boundary demand is the existing proof-relevant call-by-need protocol: the first demand evaluates once, while a current cached value or stable fault is observed without another evaluation.

Interaction supplies a nontrivial dependent instance. Above an ordinary GSLT rewrite path $p : x \rightsquigarrow y$, exact evidence is an authenticated event path whose erasure is exactly p. Exact evidence composes chronologically, precision is monotone in both operands, and a typed GSLT-IL route transports the event path while commuting with composition. Intrinsic $\texttt{Comp}\rho$ composition therefore constructs the exact composite and concatenates its occurrence history without an interior check. A many-to-one language route may identify visible endpoints while retaining distinct occurrences; conversely, a component refutation does not become a refutation of an arbitrary composite or translation without a reflection/decomposition theorem. The indexed protocol canary retains one authenticated communication for a one-field request while a two-field continuation at that index is uninhabited. Thus dependent protocols, relational nondeterminism, and gradual fallback coexist without weakening kernel conversion or making observation a gate on execution.

The same constructional law now reaches the ordinary dependent term formers. Displayed

capabilities have both an ordinary product and a dependent sum. The former combines independent function and argument evidence; the latter retains the fact that the expected type of a dependent pair's second component is obtained by substituting the first raw term into the codomain. Lambda, application, pairing, identity formation, and reflexivity are then ordinary constructional maps of these fibres. Exact premises construct the exact result judgment, whereas any suspended premise leaves the raw constructor available with suspended native evidence. Precision is monotone. A missing declaration provides the negative control: its absence refutes the proposed lambda body, so no typed lambda is manufactured, yet the raw lambda syntax is not deleted. This is the native-kernel form of graduality: evidence changes which stronger computation can be constructed, not which unrelated raw syntax may exist.

Prime's surface specification remains revisable under this architecture. The stable commitments are homoiconic language-indexed data, relational and effect-aware computation, conservative raw execution, intrinsic construction, revision-aware admission, and ergonomic access to the strongest earned native calculus. A current spelling or branded dialect point may change when doing so better realizes those commitments; the required preservation and non-collapse theorems make such a change explicit rather than silently redefining the language.

7.9 Semantic deltas forced by internalized GSLT-IL

Internalizing GSLT-IL is not merely the addition of a translation API. It changes which distinctions Prime must preserve as language semantics. The following deltas are therefore architectural requirements rather than current surface syntax commitments.

1. **Elaboration is relational before it is functional.** One authored command may have several valid elaborations, carrying distinct witnesses or proof histories. Prime must be able to retain and inspect that fibre. A profile may expose a direct elaborator only after coverage, determinacy, and history thinness have been established for the declared fragment.

2. **Languages and routes are ordinary indexed data.** Quotation records the source language, and route application changes that index. A reverse route, exact round trip, or common abstract image is evidence to construct, not an untyped string-conversion convention.

3. **Typed construction changes available computation.** Exact evidence can select a representation, direct kernel, eliminator, parallel schedule, or fused realization whose result is constructed at the indexed type. It is not restricted to pruning a raw search after the fact.

4. **Ambiguity is usable data, not an implicit error or hidden choice.** The ergonomic default may enumerate the proof-relevant alternatives. A user or program may request a named profile, refine the context until one world remains, or deliberately combine the alternatives under a declared observation discipline. No implementation may silently replace this fibre by one representative and then claim reflection.

5. **Observation and cost remain downstream.** An observer may forget route history, and a scheduler may select one execution, but neither quotient changes the underlying elaboration or execution fibre. Exact replay and higher Cost iterations consequently retain the provenance that a scalar readout can discard.

6. **Graduality is capability monotonicity, not global gatekeeping.** More precise evidence may authorize more native structure. Failure to obtain that evidence leaves unrelated raw reductions intact; staleness removes the derived capability and returns to a semantically adequate route.

The Cost_1 observation bridge makes the fifth requirement exact. A selected normalization path is first realized as a proof-relevant operational schedule; the observation container S is then the complete chronological list of wave events, including endpoints, funded-occurrence receipts, and step evidence. Work/span is a declared valuation V of that retained list, and a scheduler score Q may be derived only afterward. Concatenating paths concatenates the exact event histories and composes their work/span values sequentially. The factorization pulls back along any supplied operational realization of a Cost_1 domain object, without identifying the realization with its scalar value. The separation is strict even on valid Prime executions: running two independent funded occurrences in opposite chronological orders yields unequal receipt histories but equal work/span. Thus an equal scheduler value cannot justify replay, cache identity, or exchange of causal histories. Constructing the concrete realization for the selected rho Cost_1 object remains a separate provider obligation; the observation theorem neither assumes nor manufactures it.

This strict separation now feeds NIK at the appropriate abstraction boundary. The strongest eligible native semantics and the least sufficient observation key are different choices: the former determines how the computation runs, whereas the latter is relative to the policies a consumer requests. Work/span is therefore admitted as an executable key for a work/span-only request, but the two-order collision proves that no such key can answer a request which also observes receipt chronology. For that larger request, the dependent vector of all requested answers is the canonical least sufficient key; retaining the whole history is also sound but not required as the hot representation. A current NIK activation evaluates the admitted keyed function directly. A relevant revision change disables it and leaves the complete history available for fallback. Thus scalar equality can authorize a scheduler readout without being promoted into receipt identity, replay authority, or a weaker execution semantics.

The cache consequence has an exact information order, rather than a prescribed list of implementation fields. For two keys on the same proof-relevant state, write $k_f \succeq k_c$ when a fixed forgetting map recovers k_c from k_f. This relation is reflexive and transitive. Policy support is monotone in this order: any observation computable from k_c is computable from k_f. A key with a left-inverse decoder refines every other key, and all exact keys are equivalent in the resulting preorder. Rho's Cost^2 witness makes the order strict: the elaboration-provenance key refines the compact-term key, but not conversely. Moreover, two elaborations collide under the compact key while a future Boolean policy distinguishes them, so every proposed compact decoder must return the wrong elaboration for at least one member of that pair. This is the deterministic, proof-relevant analogue of Blackwell's comparison of experiments: more information is characterized by the decisions it can support after a common garbling, here strengthened from expected decision value to exact policy factorization and replay.

This order is now connected to revision-indexed NIK admission without making receipt selection a second semantic authority. Mere policy safety is a proposition, so it cannot by itself be the hot runner. The admitted object instead retains each keyed function as data together with its factorization proof. It is displayed over an already-admitted execution model: activation uses the same dependency-currentness witness, evaluation applies only the retained keyed function, and staleness disables the derived view while the raw execution representation remains available.

For a policy-only request, the dependent tuple of all requested observation values is a canonical least key. Every other admitted key forgets to this tuple, so it contains at least the distinctions required by the request; the canonical tuple contains no additional distinction. An exact-replay request changes the capability fibre: every admitted key is then mutually refining with the identity key on the full state. The Cost^2 counterexample exercises both boundaries. Its compact term cannot realize even the concrete policy that distinguishes two elaborations, and it cannot realize exact replay; retained elaboration provenance realizes both. Thus NIK may choose a small request-

scoped key when that is proved sufficient without ever treating a lossy cache hit as an exact historical receipt.

The same boundary now holds globally over the complete dependent Cost carrier, not only inside one fixed compact-term fibre. Global compact erasure retains the term together with its free context, bound context, and sort, while forgetting the elaboration tree. The existing Cost compiler is a section of this map. Consequently a policy is computable from global compact syntax if and only if it is invariant under every change of elaboration evidence at fixed compact syntax. More strongly,

$$\text{global compact key admits exact replay} \quad \Longleftrightarrow \quad \text{compact Cost erasure is faithful.}$$

The forward implication extracts injectivity from the replay section; the reverse implication uses faithfulness to prove that recompiling the compact term recovers the original displayed elaboration. Hence faithfulness produces an actual exact implementation codec rather than a serialization slogan. In rho's Cost^2 witness faithfulness fails, but the entire compact value remains a valid, nonconstant hot-policy answer: its runner is the identity on compact syntax. The identical key is refused when the request asks for exact replay, whereas the displayed elaboration key is admitted. A backend may therefore cache and schedule from compact syntax without pretending that the cache entry is an auditable receipt.

These conclusions are now sealed at rho's actual selected object, rather than remaining a theorem schema awaiting a first-layer configuration. The finite-support hereditary configuration is definitionally the compact output of the unconditional rho Cost_1 domain object, and the required empty-parallel representative is proved for that executor. Applying Cost elaboration to this selected output yields a genuinely nontrivial displayed fibre: it contains two distinct elaborations over one checked compact term. The complete laws-free Cost^2 package—nonfactorization, nonfaithfulness, non-subsingleton fibres, and a provenance-sensitive policy—therefore holds without premises. At every NIK revision the compact value remains an admitted policy-only hot key, compact exact replay is refused, and retained displayed provenance admits the stronger request. This closes the selected Cost^2 boundary without choosing a second-layer compact normalizer or claiming a compact selected executor.

This policy-relative boundary is stable under language transport. A conservative Cost_1 morphism already maps the retained elaboration tree structurally, preserving its ordered boundary occurrences rather than recompiling it. There is a corresponding map on the dependent compact carrier, and compact erasure forms a commuting square:

$$\begin{array}{ccc} E(L) & \xrightarrow{\text{transport}} & E(L') \\ \downarrow k_L & & \downarrow k_{L'} \\ C(L) & \xrightarrow{\text{compactTransport}} & C(L'). \end{array}$$

The square composes for successive displayed transports. Therefore a compact policy on the target can be pulled back to the source, admitted once, and run from the source compact key with exactly the same answer it would return after transport. This is the naturality law needed by NIK to move a licensed hot observation through a language route. It deliberately does not identify a freshly compiled proof tree with the transported proof tree. Rho's Cost^2 counterexample remains: the compact policy is runnable, while exact replay still requires faithful erasure or a retained displayed receipt. Functorial policy transport thus preserves acceleration without laundering provenance.

The construction applies simultaneously to a dependent family of requested policies, whose result type may vary with the policy. Each already-admitted target runner is precomposed with

compact transport, and the same currentness witness activates the whole pulled-back family; no second dependency check is introduced. Two successive pullbacks agree with observing the twice-transported retained state. The pulled-back request is explicitly policy-only: an exact-replay capability is never inferred from the family of hot functions.

The Cost specialization factors through the generic NIK theorem rather than forming a parallel admission doctrine. First the target observation family pulls back along displayed Cost state transport. Then the source compact key refines the transported target compact key through the lower horizontal map of the commuting square above. Generic admission pullback followed by this key refinement yields definitionally the same keyed executable function as the direct Cost construction. Thus semantic transport, representation refinement, current activation, and hot execution are separate proof steps but introduce no interior checker and no duplicate runner.

Semantic admission, receipt sufficiency, and profitability now form three independent axes over one current dependency world. A complete capability bundle is displayed over an already-admitted execution model: its receipt view establishes exactly the observations supported by a retained key, and an optional profitability receipt compares the complete source and target traces. Activation is inherited solely from the semantic admission. The hot runner applies the admitted semantic map and the keyed policy function; no cost proof is passed through execution. Positive and negative models prove that semantic admission plus an exact receipt cannot manufacture a false cost inequality, while semantic admission plus a true cost inequality cannot make a collapsed key support a distinguishing policy. A relevant revision change disables the whole derived bundle and leaves exact raw fallback intact.

The exact-fibre law of maximal-native selection places profitability on the correct side of the boundary. If two licensed realizations support precisely the same requested semantic capabilities, then they have identical membership in that capability request. An external cost preference may rank them only after the semantic fibre has been exposed; it cannot silently omit one and claim that the remaining realization is a stronger calculus. This preserves the genuine partial order of native capabilities while allowing ordinary cost policy to choose among already-correct realizations.

The two-stage selection itself is now formal. Filtering an exact finite request fibre by semantic maximality produces a nonempty frontier. For any declared linearly ordered cost observation, that finite frontier has a cost-minimal member, and its executable operation is still the original admitted arrow. Cost therefore refines policy only within the Pareto frontier of native structure. A concrete linear family proves the sharp negative: even when a weaker realization is assigned a lower cost, it cannot be selected because a strictly stronger eligible realization exists. Conversely, two incomparable maximal realizations with equal cost are both valid selections. They are exchanged by a fixed-point-free symmetry, so no symmetry-invariant selected element exists. A deterministic implementation must declare an additional tie policy; otherwise it retains both alternatives. Enumeration order is not semantic evidence.

The same separation applies inside typed optimization authority. There is no universal "optimization license" whose mere presence authorizes every native transformation. Instead, each hosted calculus supplies a proof-relevant judgment indexed by the exact source occurrence and the complete revision key. Deterministic dispatch binds its type and cardinality evidence to that key; storage reuse retains the actual lifetime and publication classification; and memo reuse retains both physical snapshots and the append-only derivation. The transformation-specific recognizer then proves the additional shape needed by that optimization. A retained reference and a changed snapshot therefore remain well-classified native judgments while correctly failing optimization admission. Profitability is evaluated only after this semantic boundary.

The generic optimizer now enters maximal-native selection conservatively. Observational adequacy proves that ordinary and compiled execution inhabit one semantic fibre, but it does not

by itself order their computational capabilities. A semantics-only request therefore retains two incomparable maximal faces, on which a declared profitability projection may select without pretending to discover stronger calculus structure. An explicit request for the compiled artifact instead filters to exactly the compiled face and has a unique strongest realization. Constructing that request requires both the source-and-key-indexed native judgment and the shape witness returned by the optimization's own recognizer. Thus a hosted calculus can add a stronger artifact-specific order when it proves one, while the common NIK selector never manufactures such an order from extensional equality alone.

The selected face and the abstract Prime implementation contract now form one commuting implementation triangle. Given the same source-and-key authority and recognized shape, the request-selected hot operation emits definitionally the same compiled artifact as the implementation model's exact receipt codec. This definitional agreement is interface coherence, not a correctness oracle: the independently authored adequacy theorem proves that their shared target observation equals the source observation. Both activations use the complete optimization key as their sole currentness boundary. Changing any coordinate disables both while the exact source trace remains decodable from raw fallback; failure to recognize the required shape prevents construction of the selected implementation path in the first place.

The Metamath assertion client makes this ordering executable rather than merely illustrative. Its ordinary ordered scan and immutable prepared index are two actual functions into one exact observation fibre. The equality of compiled keyed lookup with authored source lookup grounds the additional capability of the prepared face; it is not a nominal flag. Consequently the exact request has the prepared index as its unique greatest realization. Exact ordered results and observation count are retained as independent policies at the same revision, while two equal result lists with different execution faces prove that the result-only key cannot answer a face-sensitive policy. Staleness disables the selected operation and its policies together but retains the full query list and receipt. Even a deliberately adversarial cost assigning the raw scan a lower number cannot select it from beneath the semantic maximal frontier, and an ambiguous duplicate-label table cannot construct the validity evidence needed to enter the family at all. Fused substitution then composes as a separately admitted transformation after exact record selection. It is not inserted into a false global ranking with lookup realizations that have a different semantic contract.

This client now also closes the concrete implementation-policy square. One valid authored assertion occurrence compiles to an artifact state whose trace is read as a proof-relevant lookup receipt. The receipt returned by running that compiled trace is equal to the receipt returned by the uniquely strongest prepared-index policy: physical queries, ordered observations, occurrence multiplicity, and native execution face all agree. Exact-result and occurrence-count policies therefore pull back through the implementation at the same complete optimization key, and their selected source runners reduce to the artifact runners without an interior checker or dispatcher. A dependency revision change disables both policies and leaves the ordinary authored trace available; a duplicate-label source cannot construct the validity evidence needed for selection.

There is a useful asymmetry at the image boundary. Every trace produced by this compiler has the prepared face, so a fresh image-specific recognizer can soundly expose that capability. The transported result-only catalog does not acquire it automatically. This is the concrete form of the general non-surjective boundary: compilation may make a stronger native realization provable on its image, but transport preserves only declared policies. NIK must earn the new face by recognition rather than mint it from an implementation map.

7.10 Observation domains are generated, not guessed

The observation architecture is now factored into five independently varying layers: a proof-relevant indexed execution, its event history, a retained witness container S, a semantic readout $S \to V$, and an optional scheduler readout $V \to Q$. Chronological and certified-independent composition are capabilities on S, not consequences of possessing a value or a scheduler score. Failure of a partial collector omits an observed execution article; it does not remove the underlying GSLT path.

The information required by a scheduler is determined by its complete declared policy family, not by an a priori demand for either scalarity or faithfulness. For dependent policies $p \in P$ over an operational state s, let $\vec{p}(s) = (p(s))_{p \in P}$ be their answer vector. This vector has an exact universal property: it is itself sufficient, and every sufficient readout $q(s)$ admits a fixed map $q(s) \mapsto \vec{p}(s)$. Thus it is the least informative sufficient readout for that request. Reindexing to a smaller policy family projects the vector and cannot invalidate an already sufficient readout. Conversely, one collided pair separated by any requested policy refutes the whole readout. A concrete history model shows why this is sharper than "lossy is bad": length alone is a lossy but sufficient view for a length-only policy, while the identical view becomes insufficient as soon as an order-sensitive policy is added. NIK may therefore retain rich semantic state and activate a cheaper view exactly when the requested answer vector factors through it.

The Cost^2 receipt-key admission is now an explicit instance of this generic law rather than a parallel policy theory. Its retained keyed runners form the generic executable family realization, and the universal property rederives the existing refinement from every admitted key onto the request vector. For a policy-only request the passage is bidirectional: converting into generic GSLT policy admission and back preserves every keyed hot function exactly, and current activation transports with the same dependency witness. Exact replay remains additional structure rather than a consequence of this equivalence. Replay remains orthogonal: a collapsed Boolean key realizes a constant policy family while a two-state collision proves that no decoder can recover the semantic state.

An operational observation domain may contain more than the flat image of event histories. It can also contain plans built from already observed containers by chronological composition or by an independently parallel operation carrying an explicit separation witness. The proof-relevant construction generated by these clauses now satisfies its advertised universal property. Write $\mathsf{CapGen}(c)$ for membership in the generated domain and $\mathsf{Closed}(D)$ when D contains histories and is closed under the supplied licensed operations. Then

$$\mathsf{CapGen}(c) \quad \Longleftrightarrow \quad \forall D,\ \mathsf{Closed}(D) \Rightarrow c \in D.$$

The forward direction is induction over the complete construction tree; the reverse direction instantiates the quantified domain with the generated domain itself. Thus "least" is a theorem rather than an informal reading of an inductive definition.

This universal property is stable under every represented GSLT-IL route. The target collector and its declared chronological and independently parallel operations pull back to the source event stream, while the authorization family is retained unchanged. The resulting source domain is again exactly the intersection of all source domains closed under those transported capabilities. The container, readout-value, and authorization universes vary independently; transport therefore does not identify evidence with its value or turn a route license into a parallelism license.

A finite Boolean model proves strictness. Every raw history collects to `false`, while an authorized parallel composition of two such histories constructs `true`; consequently capability closure strictly extends flat history reachability. Replacing the permission family by the empty type makes the same parallel article uninhabitable. Neither commutativity, route representation, nor scheduler

preference can manufacture the missing authorization. This is the observation-level form of the wider rule that a performance opportunity is not an execution right.

This combination has substantial neighbours but no single cited predecessor is being claimed to supply it whole. Matthews and Findler compose operational models of multiple languages; Ranta's GF places typed abstract syntax between multiple concrete syntaxes; Shulman's framed bicategories distinguish maps from proarrow-like loose structure; Stay and Meredith give a higher-categorical operational model of the π-calculus; Winskel-style event structures and Nester's concurrent process histories retain causality and alternatives; dependent session types internalize value-sensitive protocols; gradual type theory studies precision and embedding–projection structure; and typed assembly, proof-carrying code, and translation validation establish check-once or type-preserving native execution. The prospective contribution here is their conjunction in one homoiconic, language-indexed, proof-relevant equipment with reflective rho nondeterminism, intrinsic dependent construction, revision-indexed admission, and several declared observation and cost valuations. Priority for that conjunction remains a literature-review question, not a consequence of terminology or title search.

7.11 Proof objects, GSLT-IL, and the bootstrap boundary

The native theory retains proof-relevant evidence rather than reducing it to truth. Proof objects may be inspected and transformed as native data; an admitted operation supplies the correct-by-construction path, while an arbitrary agent or external prover remains on the check-once boundary. The multisorted-clone structure records substitution and composition of such operations when the guest admits them. This gives a precise target for typed proof-object programming in Prime: typechecking a transformation should establish that it factors through admitted operations, after which its interior execution needs no repeat check.

Prime's connection to GSLT-IL now has two rigorously separated layers. The live Need algebra is equivalent to the single returned-fibre fragment, with step equivalence, clone equivalence, and a commuting admission square. A negative witness shows that a pending claim is not encoded by that fragment. Above it, a new two-point indexed diagram makes today's Zero and Prime distinct theory objects, maps the nonidentity Zero-to-Prime route to the existing operational translation, and proves that route application is simultaneously a GSLT-IL command step and a certificate-free NIK admission arrow. A concrete request crosses that language boundary, enters Prime Need, and then uses the admitted Need operation to flow correct-by-construction with no interior recheck. The reverse route is empty and the returned fragment still has no full command decoder. Thus the indexed bridge is nontrivial but cannot be mistaken for a derived implementation, scheduler, or arbitrary-theory language algebra.

The dialect-projection result is correspondingly split. HE, PeTTa, Zero, and Prime requirement bags are exact projections of one observed native requirement spine. This is not yet the desired authored-calculus equation

$$\mathsf{DialectRules} = \mathsf{filterMap}(\mathsf{NativeRules}, \mathsf{projection}) \mathbin{+\!\!+} \mathsf{namedQuirkDelta}.$$

The Lean target interface requires a surviving shared native rule and a quirk delta strictly smaller than the dialect calculus. A negative theorem proves why an instance cannot be extracted from bags alone: HE and PeTTa have identical observed typing-property lists but their authored base typing calculi have 22 and 21 rules. The current PeTTa tree also contains distinct 50-rule determinism and 72-rule guard presentations. Regression totals such as 132/132 or 76/76 must not be relabeled as authored rule inventories. The exact three-dialect calculus projection awaits a common finite native rule presentation and explicit maps into each actual authored calculus.

Finally, the bootstrap metalogic enforces only what its index can honestly prove. A level-$n+1$ authority may check a refinement or fidelity claim about a level-n kernel. Every edge strictly descends in level, and two-level certification cycles are impossible. This supports progressively smaller trusted bases and eventual verification of fast implementations without changing their runtime cost model. It does *not* prove same-level semantic self-consistency, and it supplies no provenance merely from the existence of a level number.

The intended reflective closure is stratified rather than circular. The List–identity package now supplies the lower direction from authored source to a formed native host with exact proof-fibre agreement between authored and native computation receipts. That authored elaboration and interpretation is also a proof-relevant GSLT-IL route: the intermediate declaration inventory is retained, yet the route is represented exactly by the direct semantic interpretation map. The converse relation, from an extensional signature to all compatible authored source worlds, is provably not representable by any single source-producing map, because interpretation forgets genuine authored distinctions. The native List–identity document is a concrete inhabitant of the forward route. Thus source interpretation can be compiled without search, while reflection remains honestly relational instead of selecting an arbitrary source world.

At level $n+1$, level-n source and its retained route evidence may themselves be quoted and inspected. The source-to-open-conversion seam is now generic: authenticated schemas instantiate through native simultaneous substitution and enter the typed conversion fibre exactly when their endpoint evidence permits. What remains for fully automatic family admission is stronger and more honest: derive the required formation and uniform preservation capability for every accepted strictly-positive declaration package, rather than postulating it or recovering it from raw support. Lower-level formation, elimination, and native computation do not depend on that later derivation, and no same-level fixed point justifies its own rules. Self-description is therefore a stratified capability of the tower rather than an extra premise of the first native kernel.

7.12 Space operators, quantified queries, and operational polarity

The fusion with Prime is not limited to placing a dependent checker beside an existing evaluator. Prime's spaces expose an algebra which the native theory must classify without turning search into definitional equality. At the raw collection level, a match has the schematic semantics

$$\mathsf{match}(S, p, t) = \{\!\{\, t[\sigma] \mid a \in S,\ \sigma \text{ unifies } p \text{ with } a \,\}\!\}.$$

Thus filtering a pattern is bounded selection, applying the result template is image, shared variables across clauses induce joins, and multiplicity remains observable until an explicit set quotient erases it. Union, intersection, difference, update, aggregation, and finite least fixed points then form natural neighbours of match. This identifies a candidate safe query fragment and a large body of database and logic-program optimization laws, but completeness of an operator basis still requires an explicit syntax, semantics, and normal-form or expressivity theorem.

Universal matching is not merely a condition-bearing macro over ordinary match. Its primitive meaning is the space-inclusion judgment

$$\mathsf{matchAll}(S, p) \quad \Longleftrightarrow \quad S \subseteq \{a \mid a \text{ unifies with } p\} :$$

every atom in the space must unify with the pattern. The finite executable contract is proof-relevant: on success it retains the complete multiset of bindings from every atom, including multiplicity; on failure it returns exactly the bag of nonmatching atoms. The success constructor remains distinct even for an empty space, so vacuous truth is not confused with evaluation failure. A result template may then be instantiated at every retained binding. Thus ordinary match

asks whether some atom has the requested shape, while universal match establishes that the entire space has that shape. Selecting one violation gives cheap failure; demanding more enumerates the violation stream. A separate operator which applies an arbitrary Boolean condition to each match is derivable from match, branching, and aggregation. It must distinguish condition value `False` from failure to produce a Boolean value; the latter is UNDETERMINED, not a counterexample and never evidence of universal success.

Logically, this coverage query has the more precise shape

$$\forall a \in S.\ \exists \sigma.\ \mathsf{Unify}(p, a, \sigma).$$

The flattened binding bag enumerates the inner existential witnesses with multiplicity; it is not by itself the dependent function witnessing the outer universal. Consequently `matchAll` is not automatically the categorical right adjoint of `match`.

This law does, however, define a genuine patterned refinement of spaces:

$$\Gamma \vdash S : \mathsf{Space}[p] \quad \Longleftrightarrow \quad \prod_{a \in S} \sum_{\sigma} \mathsf{Unify}(p, a, \sigma).$$

The dependent product retains a separately chosen unification witness for each atom occurrence; it does not require one global substitution to fit the whole space. The executable finite checker is adequate for this judgment, and the judgment is stable under reindexing of the ambient context. A concrete syntax form such as `(: &S (Space p))` is therefore coherent only when elaboration runs that checker and stores the resulting revision-keyed license. Merely parsing a type assertion is not enough. Conversely, a form such as `(in &S p)` normally reads as membership and would obscure the refinement judgment.

For an infinite intensional space, absence of a yielded violation proves nothing. When the space is the least fixed point of finitely presented generator rules, however, inclusion can be established by the pre-fixed-point principle: the pattern denotation must contain the base facts and be closed under every generator. Failure then returns a rule instance which breaks closure rather than requiring enumeration of an infinite atom. This is the induction principle for generated spaces, and it makes the native totality check a finite rule-level operation whenever the corresponding containment recognizer applies.

The Lean contract proves the finite characterization in both directions, the cache-polarity laws (removal preserves a universal fact; a bad addition returns that atom as its exact revocation witness), and the abstract least-fixed-point closure license. For logic programs the closure premise is decomposed into base facts plus preservation by every clause under every grounding. This is a semantic license, not an executable recognizer: a native fragment must still decide those quantified obligations and prove its acceptance sound, while recognizer failure yields INCOMPLETE, never refutation.

The truthful outcome depends on the admitted cardinality and generation mode:

- a materialized finite space may be scanned to exhaustion;
- a certified closure-finite rule space may first be saturated and then scanned;
- an open intensional space denotes a least fixed point and is explored on demand, but universal truth requires induction, model checking, or another admitted proof principle; and
- a bounded search returns a witness, a counterexample, or INCOMPLETE with its stated budget and coverage. Statistical or PLN evidence may decorate that last result, but it is not promoted to truth.

Cyclic derivations with an independently checked progress condition are one possible finite proof representation for inductive or μ-calculus claims; they belong at the proof-search boundary, not in ordinary evaluation.

This operator algebra is also the concrete customer for the future hyperdoctrine laws. For a context projection $\pi : \Gamma \times A \to \Gamma$, contravariant reindexing π^* has the desired adjoint chain $\exists_\pi \dashv \pi^* \dashv \forall_\pi$. Thus existential binding is the left adjoint and universal binding is the right adjoint of reindexing, not of one another and not of the raw match engine. In a proof-relevant fibre, $\forall_\pi$ returns a dependent product assigning every fibre element its witness; in the thin Boolean shadow it reduces to ordinary "all". The finite `matchAll` contract is one decision procedure for the bounded $\forall$–$\exists$ predicate above. Beck–Chevalley and Frobenius laws would then justify moving quantified matches across substitutions and joins. Those laws must be proved for the actual bag-, evidence-, and resource-sensitive fibres before the native structure is called a hyperdoctrine.

Evaluation polarity is consequently a property of a space or elaboration region, not of the top type. Computation-oriented regions may evaluate by default; knowledge-oriented regions may hold by default. The stable syntax separations are: `$x` is a MeTTa object variable; `?a` is a solvable meta-level variable; `?Unknown` is gradual imprecision; an inert top type makes no evaluation decision; language-indexed `Data` classifies held syntax; and `@/!` are the quote/run introduction and elimination forms. A future universal-match syntax and type-directed optional arguments are elaborator choices, not additional kernel equalities. Position-free arguments are sound only when their expected types select a unique placement; otherwise names or tags are required.

7.13 Typed weighted temporal control

Three neighboring algebras must remain distinct. A graded clause changes the weight of an authored reduction and uses a commutative semiring, so alternative matches can be accumulated independently of enumeration order. A policy grades and reorders already-authorized occurrences using a quantale; its sequential product need not commute, because route order may matter. A modal μ-calculus objective describes temporal acceptance, and a parity progress certificate can justify a cyclic liveness argument. None of these objects authorizes a new reduction edge.

The generic controller preserves every live occurrence exactly. Therefore every finite run is sound, and any two controllers which both exhaust the same additive search have the same answer bag. Liveness is intentionally not policy-independent: a legal depth-first controller can starve an answer which a breadth-first controller reaches. Fairness, recurrence, and eventuality must be separate ν/μ objectives with checked progress evidence. Fixed-point bodies are admitted only after the existing positivity check; the concrete formula $\mu X.\neg X$ is rejected rather than passed to a parity authority.

Prime's semantic CwF now contains policy, temporal-objective, and weighted parity-automaton values with their carriers visible in their types. The default Need policy elaborates to exactly the pre-existing breadth-first scheduler. PLN uses the already established order-dual parallel-evidence quantale directly: revision becomes quantale multiplication, retaining positive and negative support. When a graded `where` clause specifically requires a scalar, an explicit projection uses the already-proved finite-evidence equality

$$\mathsf{strength}(e)\,\mathsf{confidence}_\kappa(e) = \frac{e^+}{e^+ + e^- + \kappa}.$$

Equal strength with unequal evidence volume yields unequal projected propensity, so confidence has not been erased. Cost, weakness, and other carriers remain separate quantale instances and may be related only by stated morphisms or product constructions.

The Boolean and quantitative accounts now share a proved boundary rather than parallel folklore. Proposition-valued quantale semantics is exactly the ordinary modal semantics, constructor by constructor, including fixed points. Consequently the existing polarity theorem for quantale-valued satisfaction specializes to Boolean positivity without a second structural induction. A positive fixed-point body denotes a Mathlib monotone order homomorphism on the powerset of states; its impredicative μ and ν clauses are respectively the Knaster–Tarski least and greatest fixed points. On a finite state carrier, both stabilize after at most the carrier cardinality: iterate from the empty set for μ and from the universal set for ν. These finite approximants are the semantic ranks required by the remaining parity-game adequacy proof.

Two boundaries remain open. An independently specified Prime concrete grammar and elaborator must be proved to denote these semantic policy values; semantic inhabitation alone is not a full internal-language theorem. The positive modal μ-calculus compiler is now executable: it separates ambient observations from fixed-point back-edges, produces a finite verifier/falsifier arena, proves every generated pointer in bounds, proves that every binder back-edge reaches an actual introducing fixed-point node, and connects an executable finite model pointwise to the original denotational objective. Winning strategies include local move validity, so an illegal selected edge cannot win merely because it generates no play. A proof-side semantic table is structurally aligned with the node table, and finite least-entry and greatest-exit ranks connect fixed-point truth and refutation to the approximants needed by the parity proof. The unit-weight embedding places any such arena in the existing quantale-weighted automaton type. As a nontrivial witness, the existing $\nu Y.(\mu X.O \vee \langle a\rangle X) \wedge [a]Y$ recurrence objective compiles to a nine-node cyclic arena; its denotation holds on a one-state observed loop, and an executable progress certificate proves its selected strategy parity-winning. The generic adequacy theorem

$$s \models \varphi \quad \Longleftrightarrow \quad \text{the verifier wins the compiled game from } (s, \varphi)$$

remains to be proved. Thus concrete checked games are valid authorities, while arbitrary positive formulas are not yet promoted solely by compilation.

7.14 Categorical dimensions and the ultrainfinite boundary

Several categorical structures coexist, at deliberately different strengths. At dimension one, contexts and substitutions form a guest-chosen category; claims and accepted evidence reindex contravariantly. Language codes and structural translations form another category; the Data family varies covariantly and its Grothendieck total category has chosen strongly cocartesian push-forward lifts. These variances must not be conflated. The current Data theorem is opfibration-shaped: without contravariant pullback it is not a full fibration, and without packaged cleavage coherence it is not yet advertised as a split opfibration.

The neutral contextual evidence structure is the beginning of a proof-relevant indexed doctrine, not yet a full hyperdoctrine. A full hyperdoctrine would add logical structure in each fibre, adjoints to reindexing for quantifiers, and Beck–Chevalley/Frobenius coherence. Generic predicate-fibration and Beck–Chevalley infrastructure exists separately, while the native theory has a CwF-style comprehension and substitution spine. Instantiating the former on the actual native Pattern doctrine, with proof-relevant bags and resource-sensitive variants handled correctly, remains a theorem-level integration task. Proof erasure already has a precise categorical comparison: proof-relevant set-evidence reflects to thin closure through an adjunction, and indexed erasure yields an optional π-institution. Neither result makes thin consequence the definition of NIK.

At dimension two, an execution route retains its authored steps as data. A generating 2-cell compares two routes with the same endpoints; vertical composition and left/right whiskering freely

build larger comparisons. Concrete naturality diamonds compare "reduce then transport" with "transport then reduce", and Prime has a comparison between direct and lazy Need routes. This is useful proof-relevant two-dimensional syntax, but not yet a bicategory: unit and associativity coherence for 2-cells, interchange, and the appropriate quotient or higher witnesses remain open. In particular, the raw generated syntax is proved not to collapse a reflexive vertical composite definitionally, which prevents an unearned coherence claim.

The dependent second-*Sigma*-beta boundary above is a small concrete reason this extra dimension matters. Two substitution orders can have the same typed endpoints and the same transported term reduction while retaining differently shaped derivations of type conversion. Endpoint equality is therefore too thin, and literal equality of proof trees is too strict to assume. A bicategorical completion should relate the trees by a generated coherence 2-cell; an $(\infty, 2)$-completion should retain transformations between such cells rather than forcing all coherence proofs into one quotient. The present strict squares embed as the degenerate cells where that comparison is judgmentally immediate.

The candidate comparison is now checked rather than merely heuristic. Two reflexive conversion histories over the same endpoint have different constructor counts, so no endpoint-only readout can reconstruct even that coarse receipt observation. The histories are not equal as proof trees, yet a free authored generator relates them without identifying them. Conversely, the raw generated cell syntax is not definitionally left-unital. These three controls rule out both the endpoint quotient and literal receipt equality as a universal solution, while also preventing the free syntax from being presented prematurely as a bicategory. For dependent second-Σ-beta, the two substitution routes are aligned to literally common type endpoints and become one proposed generator of this free syntax; semantic adequacy and the unit, associativity, interchange, substitution, and observation coherences remain the selection gate.

The common layer beneath these candidates has now been extracted. A raw one-cell base supplies objects, retained one-cells, and binary composition; a parallel generator family then freely builds comparison cells by reflexivity, vertical composition, and left/right whiskering. For every target family that interprets those five constructors, the type of structural extensions is contractible. This is a relative initiality theorem over the fixed retained one-cell base, not a claim that the one-cells already satisfy category laws. The existing GSLT generated cells are exactly equivalent to this common syntax, as are the dependent structural-conversion cells. Both equivalences preserve complete constructor trees in both directions. The dependent second-Σ-beta comparison is therefore an attached two-generator in the same raw language as an authored operational optimization diamond.

Initiality and reflection are now separated at theorem level. Any interpretation of primitive comparison generators extends uniquely over the complete raw cell syntax, but uniqueness of that extension does not make it faithful. A fibrewise retraction of the primitive-generator interpretation induces an injective map on all generated cells, including their vertical and whiskering history; a fibrewise equivalence induces an exact equivalence of complete cells. Conversely, identifying two distinct primitive generators provably identifies two distinct complete cells even though the corresponding extension space remains contractible. Thus a GSLT-IL realization may advertise structural interpretation unconditionally, structural reflection only with split evidence, and exact lossless transport with invertible evidence. This is also the selection boundary for a maximal native kernel: a faster face may discard proof history only when its declared observation factors through that discarding, while a proof-sensitive face must retain or reconstruct it.

That selection boundary is now executable as an exact NIK request rather than only stated as a criterion. A split generator map realizes every dependent policy family by reconstructing the complete source cell. A non-split map which collapses two authored receipts still realizes the

constructor-shape policy, because structural recursion preserves constructor count, but it cannot realize the family containing a receipt-identity policy. Placing the collapsed and retained maps in one admitted two-member native family therefore leaves both eligible for a shape request and removes the collapsed member from the exact receipt request. The strongest receipt selection executes the original retained admission arrow and factors onto the complete requested answer vector; it neither inserts a checker nor guesses the erased receipt. Lossiness is thus judged relative to an explicit consumer family, while accountability-sensitive distinctions are conserved whenever any declared consumer can observe them.

Primitive-generator loss is not the only boundary. A higher completion may also identify composite cell trees, for example a comparison preceded by a reflexive vertical cell. The common free-cell layer now carries two generic structural observations: authored-generator count ignores reflexivity and is invariant under vertical units and reassociation, whereas raw-constructor count retains the inserted unit. On Prime's concrete dependent receipt comparison, the generator-count key identifies the unit-padded and unpadded cells and executes the corresponding unit-insensitive policy. The same key is provably insufficient for the family that also requests raw construction history; in fact, every key identifying that pair is insufficient for the stronger family, while the retained cell itself supports both observations. Thus unit coherence is neither globally denied nor silently imposed: it is a capability-indexed observation. A strict or bicategorical compression may be selected for consumers invariant under its equations, while a history-, accountability-, or repair-sensitive consumer retains the raw computad or an equivalently faithful key. This supplies a theorem-sized compatibility gate for any later bicategorical or $(\infty, 2)$ completion without choosing that completion prematurely.

Reassociation supplies an independent control against mistaking one statistic for the gate itself. The left- and right-associated Prime receipt trees have equal authored-generator counts and equal total constructor counts, but their raw constructor shapes differ. A third policy requests that shape, and every key identifying the reassociated pair is refused for the enlarged family. Interchange supplies the parallel control. Horizontally composing two dependent receipt cells has two raw readings: whisker the left comparison across the source of the right and then the right across the target of the left, or perform those transports in the opposite order. The two readings have the same endpoints, authored-generator count, and total constructor count, but different raw shapes. Consequently the generator-count key is an executable sufficient readout for a work-like request and is provably insufficient for a request that includes construction history. The exact raw cell remains sufficient for both. A strict interchange law is therefore an earned observational compression, not a global equation imposed on Prime's proof-relevant receipts. The corresponding revision-indexed NIK instance constructively admits the generator-count runner for the small request and the retained-cell runner for the full request, while proving that no generator-count admission can inhabit the full request. Current activation invokes the stored keyed function; changing its dependency revision prevents activation and leaves the complete horizontal cell available for gradual fallback.

Substitution supplies a second, independent maximal-native boundary. A map on the one-cell base does not by itself transport proof-relevant cells: each primitive comparison generator must also have a target generator with the substituted endpoints. Such a generator-naturality witness extends uniquely by structural recursion over reflexivity, vertical composition, and both whiskerings. For Prime's dependent conversion receipts, the resulting nonidentity closing substitution preserves authored-generator count, total constructor count, and complete raw shape. A target family with an empty generator fibre cannot construct the witness. NIK can therefore place exact source retention and strict target-cell construction in one recognized family, select the latter as the unique strongest face for the target-cell request, and activate it only while its declaration revision is current. Staleness removes that native activation without consuming the complete source cell.

One strict transport and a composable family of strict transports are different capabilities. The proof-relevant root interface supplies actions of renaming and substitution, but functorial use additionally requires identity and composition coherence for those actions. This requirement is not derivable from their types: a checked countermodel leaves evidence fixed when source and target arities agree and increments it when they differ. It preserves identity substitution, yet a round trip through a different arity increments twice while the composite identity increments zero times. Hence a native kernel may earn one-step generator transport without earning fusion of successive substitutions.

The canonical retained Prime root now earns the second license. Its authored root witnesses are propositions, so proof irrelevance supplies the root action laws; dependent induction then lifts those laws through every structural constructor. Identity and composite substitutions preserve the complete primitive-step receipt and the complete reflexive–symmetric–transitive conversion tree, including intermediate terms, binder liftings, and beta receipts. The result is deliberately heterogeneous equality: substitution changes indexed endpoints propositionally, while retaining exactly the same proof-relevant construction. Chaining kernels therefore require two licenses: this now-proved retained-root coherence, and a functorial generator action for the authored comparison family. The latter is not manufactured by the former, but the tagged Prime receipt family proves it independently. The two laws then lift through the complete free cell and establish, for every dependent cell C, the heterogeneous equality

$$\mathsf{Sub}_\tau(\mathsf{Sub}_\sigma(C)) \simeq_h \mathsf{Sub}_{\tau\circ\sigma}(C).$$

This is not endpoint agreement: every primitive comparison generator, intermediate vertical node, and left or right whiskering is retained. A nontrivial close–then–reopen substitution supplies the positive control. A generator action whose local maps stamp each arity change supplies the negative: it has every individual map but cannot inhabit the functoriality interface.

NIK can therefore expose a three-face request-local family: exact source retention, successive strict transport, and one direct composite transport. This is now an instance of a generic theorem. Given two admitted stages $f : A \to B$ and $g : B \to C$, a direct operation $d : A \to C$, and a declared observation $o : C \to O$, the square

$$o(d(x)) = o(g(f(x)))$$

generates the three-face maximal-native family. The request for direct composition has the direct face as its unique strongest member; admission at a dependency revision stores that operation, current activation runs it without rechecking the square, and loss of dependency currentness makes the operation unavailable. Identity observation gives exact result preservation. A checked parity counterexample shows why equality under a coarser observation must not be advertised as exact composition.

Prime's dependent receipt instance requires a more careful exact observation. Successive and direct substitutions have propositionally transported indices, so their raw Lean packages need not be propositionally equal. The observation retains the literal source and quotients the complete target cell only by heterogeneous equality. Quotient exactness proves that two observed target classes are equal if and only if the entire dependent cells are heterogeneously equal. Thus the comparison forgets index-transport bureaucracy but neither receipt provenance nor cell-constructor history. The instance constructor requires both the retained-root and generator laws. This separation lets NIK choose the strongest proved kernel rather than silently upgrading local correctness into a global functor law. The ordering records semantic constructional capability, not predicted speed: profitability remains a separate measurement-backed admission.

More generally, for any nonempty retained state, a readout supports a dependent policy family if and only if every two states in one readout fibre agree under every requested policy. The forward direction is constructive. The reverse existence theorem is classical only because a total runner must assign answers to unreachable keys; when the readout carries a concrete section, the full equivalence is constructive. NIK therefore stores an executable realization and its agreement proof. It does not invoke the extensional theorem as a runtime synthesis oracle. This is the precise distinction between a mathematical criterion for the strongest admissible compression and an actual native kernel implementing that compression.

This construction is computad-shaped in the precise modest sense needed here: generators and their globular boundaries are retained, and structural recursion supplies the universal interpreter. Street's computads, Burroni's polygraphs, and the modern inductive account of Dean–Finster–Markakis–Reutter–Vicary are the direct higher-categorical precedents. Their fuller free strict or weak ω-categorical completions are not being claimed. Prime has fixed the common input to such completions while leaving their equations or higher witnesses reviewable.

The present identity theory is intensional dependent type theory with native identity formation, reflexivity, substitution, and a dependent eliminator; it does not postulate univalence, higher inductive types, or an extensional collapse of paths. A homotopy interpretation should therefore be added as a model of the same intrinsic syntax, or as an explicitly stronger hosted calculus, rather than by silently identifying proof-relevant operational paths. In such a model, univalence can classify equivalences of data fibres, higher inductives can present quotient-like spaces while retaining their path constructors, and truncation can state exactly which proof or occurrence distinctions an observer forgets. The strict outer presentation and proof-relevant operational receipts remain useful even when the inner semantic fibres are homotopical.

For GSLT-IL specifically, represented tight routes and ambient relational loose routes already motivate the more structured equipment layer: representable routes earn companion and conjoint cells, while genuinely relational routes do not acquire a fictitious global elaboration function. A future higher completion should enrich this interface, not replace it. Its homs would become ∞-categories and its current cells would be recovered by homotopy 2-truncation. Riehl–Verity's virtual equipment of modules and Ruit's ∞-equipments provide direct precedents for this shape; Riehl–Shulman's Segal and Rezk types provide an internal language for coherently associative directed composition. Two-level type theory supplies the complementary stratification: a strict outer layer can organize syntax, stages, and coherence diagrams while a fibrant inner layer supports homotopy-sensitive identity and univalence. Operational histories, evidence occurrences, and receipts are not identified merely because their current observations are equivalent; any such forgetting remains an explicit truncation or quotient.

"Ultrainfinite" names a two-variance ambient programme, not a completed $(\infty, 1)$- or $(\infty, 2)$-category. Filtered compact stages map into an ambient GSLT (the Ind-like growth direction), while a cofiltered atlas projects the ambient object out to compact perspectives (the Pro-like observation direction). Compact probes into a supplied filtered colimit factor through a finite stage; recovery of the ambient object from all shadows is a separate optional limit property. Ultrafilters then state perspective-relative verdicts, with principal collapse and reindexing laws, without turning a perspective into the whole object. The higher-categorical endgame would organize routes, transformations, transformations between transformations, and their coherences; the present formalization stops honestly at finite routes and generated 2-cells.

The bootstrap tower is orthogonal but compatible: its level index orients authority edges strictly downward, while the ambient charts organize growth and observation. One says which level may check which lower level; the other says how finite stages and perspectives relate to an ambient system. Neither entails semantic consistency, reconstruction, or provenance. Their common design

rule is that finite, carried data—proofs, routes, cells, stamps, and receipts—must remain available when operational use needs it, while its proof-erased or observational shadow is taken only by an explicit map.

The present bootstrap implementation is genuinely natural-number indexed. Its lowest contract uses a finite index below the host, its tower is assembled at successor levels, and the native quotation and universe witnesses use adjacent successors. A transfinite generalization is therefore not a global textual replacement of `Nat`. It should first extract a weak descent interface indexed by an arbitrary type with a well-founded strict relation:

$$\mathsf{LowerContract}_{<}(h) = \sum_{\ell} (\ell < h) \times \mathsf{Statement}(\ell).$$

Strict descent, absence of self-edges and two-cycles, and well-founded induction belong to this layer. Adjacent quotation and recursively generated universe levels require a separate successor or ordinal-recursion capability. The existing natural-number tower must then be recovered as a conservative instance before ordinal or Tarski–Grothendieck indices are installed. No density or arbitrary interpolation follows from well-foundedness, and no transfinite index supplies a semantic model or consistency proof by itself.

The descent-only interface has now been extracted and proves no self-edge, no two-cycle, composition of descent, and absence of infinite descent over any partial order with well-founded strict order. It also proves `StrictlyBelow n` equivalent to `Fin n`, rules out dense refinement, exhibits lexicographic administrative coordinates, and supplies an ordinal witness. This closes the index-independent descent core only; migrating the successor-indexed quotation and universe tower remains open.

7.15 Status boundary and division of exposition

The current consolidation unit is `NativePrimeTheoryCrown.lean`. It is deliberately an intentional leaf gate rather than an import of the repository-wide umbrella. A successful build of this leaf establishes that the named Prime, GSLT-IL, NIK, observation, and Cost interfaces typecheck in one dependency graph. It does not by itself establish production-umbrella promotion, a concrete runtime refinement, or the absence of dependencies on a still-active provider proof.

Production promotion should preserve four separately checkable layers. The first is the language-independent and Prime-native mathematical core: typed routes, proof-relevant execution, intrinsic dependent construction, gradual fallback, and revision-indexed admission. The second is the Cost annex: abstract receipt and valuation laws first, followed only after its seal by the selected rho Cost_1 provider and Cost_2 nonfactorization corollaries. The third contains guest instances such as Metamath, GDL, and higher-order relational learning. The fourth is the abstract implementation-model contract and its eventual CeTTa refinement. Each layer needs its own positive and refusing controls and its own build gate. In particular, a concrete Cost provider must not become an accidental prerequisite for staging unrelated Prime or GSLT-IL theorems, and a checked mathematical crown must not be cited as evidence that the production C path already realizes it.

At this revision the combined Lean modules for the NIK metalogic and native type-theory derivation build together, their public theorem audit uses only the project-permitted foundational axioms, and the named derivation obligation count is zero. In addition, finite maximal licensed-calculus selection, a concrete Zero-to-Prime indexed language-changing route, capability-level dialect projections, a language-indexed Data family with chosen strongly cocartesian push-forwards

and four admission capabilities, the first exact HE-to-Prime Data delta, and a bounded Lean/Lean-minus NIK guest skeleton are now formalized. The regular-normalization line additionally has a presupposition-complete least typing relation, a global presupposition theorem, three independent negative premise ablations, and a structural renaming and substitution theory. It also has a scope-aware Kripke function candidate, an argument-indexed dependent extension, and a proved nonconstant codomain witness. Its semantic-environment layer now also has candidate-respecting substitutions, context comprehension, projection, pairing, and the two substitution beta laws. A stronger coherent layer proves that pointwise environment reduction acts through substitution, preserves context realization, and leaves the selected dependent candidate invariant. Exact beta and pair-projection head expansion is also proved at this coherent semantic layer, together with an erasing-omega counterexample to arbitrary reverse-reduction closure. The subsequent realized-context layer conservatively embeds those coherent candidates and proves contextual extension, weakening, and reindexing without assigning candidate semantics to malformed environments. Its dependent function candidate, including the exact constructor expansion laws and nondegeneracy canaries, is also proved. Above that layer, a proof-relevant semantic universe interprets the four current type-code formers, semantic contexts retain their candidate meanings, and semantic substitution pulls type meanings back through dependent context extension. Its conversion boundary is uniformly normalizing in every realizing environment and is itself stable under semantic substitution. The universe is coherent under conversion: constructor heads are conversion-invariant, and convertible codes have extensionally equivalent meanings, including dependent codomains compared after semantic transport across equivalent bases. The simultaneous formation-and-membership fundamental lemma is proved on top of this coherence, yielding semantic realization of every regular context and strong normalization of every subject admitted by a full regular judgment; omega is the negative non-admission witness. The following claims remain outside that closure: Prime-internal morphism and admission terms satisfying the four internalization gates above; a production full-DTT Pattern checker; automatic extraction of dialect bags from live realizations; the exact authored-calculus projection-plus-delta theorem; the intrinsic–Pattern Pure typing reflection theorem for a regularized Pattern boundary beyond the now-proved syntax, proof-fibre, cofinite-renaming, and forward-typing results; the full arbitrary-theory Prime–GSLT-IL language algebra; discharge of the external Lean4Lean/Lean4Less guest package; automatic recognition of every calculus license; and verified staged transport down to the production C realization. The proposed complete space-operator basis, production universal-match realization, cardinality-mode recognizers, cyclic-proof authority, and actual quantifier adjunctions remain design targets. The finite proof-relevant universal-match contract, cache polarity, abstract closure license, and a one-sided closure-recognizer interface are now Lean theorems; they do not yet constitute the native operator or a complete recognizer family.

This paper is the home of the unified kernel/native-theory architecture. `ProofGSLT.tex` retains the narrower mathematics of proof presentations, open derivations, checked DAGs, substitution, transport, and wire formats. `CeTTa.tex` records the implementation consequences: direct typing, admit-once fast paths, boundary-only receipts, typed dispatch, and compiled authority realizations. Neither companion should redefine NIK as its own special case.

8 Conversion discipline (the central hazard)

The single biggest architectural risk is identifying kernel conversion with general MeTTa evaluation. We hold five lines: (A) decidable conversion $\neq$ general MeTTa eval — the kernel's $\beta/\eta/\delta/\iota$ are its own bounded MeTTa functions (resource caps in the spirit of Megalodon's `BetaLimit`);

(B) MeTTa's many-valued/nondeterministic typing vs. a functional DTT judgment — the kernel commits to a functional infer/check; (C) grounded/FFI values are a trusted boundary — grounded equality is not kernel reduction; (D) universe consistency is not optional forever, because proofs *about* MeTTa are a goal; (E) the kernel must be small and separable — a pure checking slice, not entangled in effectful evaluation.

9 Open questions (kept deliberately open)

Resolved or refined by the realized architecture (§6): the `VecN`/index-domain question (indices are an arbitrary telescope under `DIndG`); the first dependent-typing fragment ($\lambda\Pi$ + `DIndG` indexed inductives, predicative); the trust boundary (it is signature *admission*); and the convergence target (the kernel is its own $\lambda\Pi$-LF, with `PureKernel`/Megalodon as templates and the Lean-verified LeaTTa as the runnable reference floor, rather than `PureKernel` itself).

Live and deliberately open: (1) the **universal subject-reduction / confluence / termination metatheorem** for the generated ι-rules — the single open soundness obligation of the realized LF calculus, now a Lean target (the toolchain is unblocked at v4.31); (2) **engine-level term sharing** (hash-consing / DAG / normalization-by-evaluation in CeTTa) for deeper scaling, of which admit-once batch checking is the cheaper architectural half already in hand; (3) a **checked conservative-definition principle** so hosted definitional equations are derived rather than postulated; (4) a **fully general derived-rule library** (beyond specific checked replays); and (5) **breadth** — hosting further core logics as LanguageDefs (fuller HOTG via Megalodon, then Lean-core/CIC) over the now-shared, now-affordable waist.

For the generalized architecture of §7, the main open goals are complementary rather than replacements for that calculus theorem: (6) a full executable native Pattern DTT with exact parity to its declarative evidence doctrine; (7) live capability and parity witnesses for each advertised Zero/Prime/HE/PeTTa bag entry; (8) the intrinsic–Pattern Pure typing reflection theorem after making Pattern-side presuppositions intrinsic (raw `PureHasType` is now proved too permissive for equivalence, while the regular intrinsic side is globally presupposition-closed); (9) the indexed Prime–GSLT-IL language-manipulation algebra beyond the now-proved arbitrary validated-language Data family and contextual hygiene, especially authority-preserving action on language presentations themselves and executable dialect round trips; (10) recognizers and adequacy proofs that populate the finite maximal-calculus license layer for real guests; (11) the exact authored-dialect projection-plus-quirk-delta instances; (12) discharge of the Lean4Lean/Lean4Less guest package; and (13) a concrete staged refinement chain from the mathematical authority to the production realization; (14) the safe space-operator basis and its licensed relational rewrites; (15) native realization and differential qualification of the now-formal finite, intensional, and bounded universal-query contracts, including fragment-complete closure recognizers; and (16) migration of the current bootstrap tower onto the now-formal descent interface, while keeping successor-sensitive quotation and universe structure as an explicit stronger capability. None of these gaps licenses weakening exact proof-fibre parity, importing cartesian structure into resource guests, or claiming semantic self-consistency from the bootstrap index.

References (local anchors)

- G. Smolka and C. E. Brown, *Introduction to Computational Logic*, Lecture Notes, Saarland University, 2014. (DTT curriculum spine.)

- *metta_proof_kernel_grounded* (AIrxiv, 2026): framework core, object-theory profiles, conversion discipline, hash imports.
- Megalodon: `repos/megalodon-1.13/src/mathdata.mli` (the HOTG `tp/tm/pf` + `check_doc` core); `examples/` (Egal/Mizar preambles); `experiments/01_basics.mg` (seed).
- Lean `PureKernel/README.md` and modules (the DTT reference/template).
- `PureKernel/RegularNormalizationBoundary.lean`, `RegularNormalizationAlgorithm.lean`, and `RegularReducibility.lean`, and `RegularKripkeCandidates.lean`, and `RegularCandidateSemantics.lean`, and `RegularCandidateCoherence.lean`, and `RegularHeadExpansion.lean`, and `RegularContextualCandidates.lean`, and `RegularContextualPi.lean`, and `RegularContextualSigma.lean`, and `RegularAccessibleConversion.lean`, and `RegularContextualIdentity.lean`, and `RegularStructural.lean`, and `RegularSemanticUniverse.lean`, and `RegularSemanticCoherence.lean`, and `RegularFundamental.lean` (exact conversion boundary, executable accessibility-driven normalizer, and the renaming-stable reducibility layer with substitution counterexamples, scope-aware function candidates, and the first genuinely argument-sensitive codomain, followed by semantic context comprehension, candidate-respecting substitution, reduction coherence of dependent environment indices, and exact constructor head expansion with its negative arbitrary-reversal control, followed by the conservative realized-context candidate interface and its genuinely dependent function and pair candidates, together with the accessibility guard required for semantic conversion, the scoped normalization semantics for identity witnesses, the global presupposition and specification-ablation theorems, and the proof-relevant substitution-stable semantic universe together with extensional uniqueness of semantic meanings under conversion, the simultaneous fundamental lemma, and strong normalization of every fully regular judgment).
- `hyperon/metamath/pverify` (MeTTa-hosted checker precedent).
- J. Meseguer, "General Logics," 1989 (proof calculi and covariant translation of proof objects; stronger conservativity is separate).
- B. Ahrens and P. L. Lumsdaine, "Displayed Categories," 2019 (indexed object/morphism families without an assumed lifting property).
- C. A. Hermida, *Fibrations, Logical Predicates and Indeterminates*, University of Edinburgh, 1993 (indexed proof families and substitution as fibred transport).
- B. Jacobs, "Comprehension Categories and the Semantics of Type Dependency," *Theoretical Computer Science* 107(2), 1993, 169–207 (dependent fibres, comprehension, and cartesian reindexing).
- R. Street, "Categorical Structures," in *Handbook of Algebra*, vol. 1, 1996, 529–577 (computads and free higher-dimensional categorical structure generated from globular data).
- A. Burroni, "Higher-Dimensional Word Problems with Applications to Equational Logic," *Theoretical Computer Science* 115(1), 1993, 43–62 (polygraphs as higher-dimensional presentations of rewriting and equational reasoning).

- C. Dean, E. Finster, I. Markakis, D. Reutter, and J. Vicary, "Computads for Weak ω-Categories as an Inductive Type," *Advances in Mathematics* 450, 2024, 109739 (explicit inductive free words and universal weak higher-categorical completion).
- D. Ara, A. Burroni, Y. Guiraud, P. Malbos, F. Métayer, and S. Mimram, *Polygraphs: From Rewriting to Higher Categories*, Cambridge University Press, 2025 (low-dimensional coherence, rewriting, and the homotopy theory of polygraphic presentations).
- M. Shulman, "Framed Bicategories and Monoidal Fibrations," *Theory and Applications of Categories* 20 (2008), 650–738 (companions, conjoints, and map-like structure inside a double-categorical ambient semantics).
- E. Riehl and D. Verity, "Kan Extensions and the Calculus of Modules for ∞-Categories," *Algebraic & Geometric Topology* 17(1), 2017, 189–271 (modules and representables in a virtual ∞-equipment).
- J. Ruit, "Formal Category Theory in ∞-Equipments I," arXiv:2308.03583, 2023 (double ∞-categories supporting formal higher category theory).
- E. Riehl and M. Shulman, "A Type Theory for Synthetic ∞-Categories," *Higher Structures* 1(1), 2017, 147–224 (directed shapes, Segal types, Rezk types, and covariant fibrations).
- D. Annenkov, P. Capriotti, N. Kraus, and C. Sattler, "Two-Level Type Theory and Applications," *Mathematical Structures in Computer Science* 33, 2023, 688–743 (strict outer metatheory together with an inner homotopy type theory).
- The Univalent Foundations Program, *Homotopy Type Theory: Univalent Foundations of Mathematics*, 2013 (identity types as paths, univalence, and higher inductive types).
- P. LeFanu Lumsdaine, "Weak ω-Categories from Intensional Type Theory," *Logical Methods in Computer Science* 6(3:24), 2010, 1–19 (the iterated identity types of a type carry a weak ω-category structure).
- B. van den Berg and R. Garner, "Types Are Weak ω-Groupoids," *Proceedings of the London Mathematical Society* 102(2), 2011, 370–394 (the canonical higher structure generated by iterated identity types is groupoidal).
- C. Cohen, T. Coquand, S. Huber, and A. Mörtberg, "Cubical Type Theory: A Constructive Interpretation of the Univalence Axiom," TYPES 2015, LIPIcs 69, 2018 (computational paths, composition, and univalence).
- E. Cavallo and R. Harper, "Higher Inductive Types in Cubical Computational Type Theory," *Proceedings of the ACM on Programming Languages* 3 (POPL), 2019 (a schema of indexed cubical inductive types and its canonicity theorem).
- M. Abbott, T. Altenkirch, and N. Ghani, "Containers: Constructing Strictly Positive Types," *Theoretical Computer Science* 342(1), 2005, 3–27 (shape–position normal forms and their functorial semantics).
- P. Morris, T. Altenkirch, and N. Ghani, "Constructing Strictly Positive Families," CATS 2007, 111–121 (strictly-positive indexed families and their indexed-container normal forms).

- T. Altenkirch, N. Ghani, P. Hancock, C. McBride, and P. Morris, "Indexed Containers," *Journal of Functional Programming* 25, e5, 2015 (reduction of strictly-positive families to indexed containers and generic programming over the normal form).
- P.-É. Dagand and C. McBride, "Elaborating Inductive Definitions," arXiv:1210.6390, 2012 (source inductive declarations elaborated into an internal universe of datatype descriptions).
- J. Ruit, "Formal Category Theory in ∞-Equipments II: Lax Functors, Monoidality and Fibrations," arXiv:2408.15190, 2024 (lax morphisms, monoidal structure, and internal fibrations for ∞-equipments).
- J. Matthews and R. B. Findler, "Operational Semantics for Multi-Language Programs," POPL 2007, 3–10 (semantic composition of languages and explicit interoperation boundaries).
- D. I. Spivak, "Functorial Data Migration," *Information and Computation* 217, 2012, 31–51 (schema-indexed instances and compositional transport between them).
- T. J. Green, G. Karvounarakis, and V. Tannen, "Provenance Semirings," PODS 2007, 31–40 (bag multiplicity and derivation-sensitive provenance as distinct projections of retained annotated computation).
- R. Li, H. Grodin, and R. Harper, "Directed Proof-Relevant Logical Relations in Simplicial HoTT," arXiv:2607.08154, 2026 (directed proof-relevant transport and the separation of reduction from horizontal representation evidence in a higher setting).
- A. Ranta, "Grammatical Framework," *Journal of Functional Programming* 14(2), 2004, and *Grammatical Framework: Programming with Multilingual Grammars*, 2011 (typed abstract syntax, concrete linearizations, and translation through a shared interlingua).
- J. G. Siek and W. Taha, "Gradual Typing for Functional Languages," Scheme and Functional Programming 2006; M. S. New and A. Ahmed, "Graduality from Embedding–Projection Pairs," ICFP 2018 (precision-indexed interoperability and its semantic graduality law).
- P. Cousot and R. Cousot, "Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints," POPL 1977, and "Abstract Interpretation Frameworks," *Journal of Logic and Computation* 2(4), 1992; P. Cousot, R. Cousot, and L. Mauborgne, "The Reduced Product of Abstract Domains and the Combination of Decision Procedures," FoSSaCS 2011 (sound precision orders, best abstractions, and reduced combinations).
- D. Blackwell, "Comparison of Experiments," in the *Proceedings of the Second Berkeley Symposium on Mathematical Statistics and Probability*, 1951, 93–102, and "Equivalent Comparisons of Experiments," *Annals of Mathematical Statistics* 24(2), 1953, 265–272 (informativeness ordered by post-processing and the decision problems an observation can support).
- R. Milner, "LCF: A Way of Doing Proofs with a Machine," 1979, and M. Gordon, R. Milner, L. Morris, M. Newey, and C. Wadsworth, "A Metalanguage for Interactive Proof in LCF," POPL 1978 (abstract theorem values and soundness by construction).

- G. Nelson and D. C. Oppen, "Simplification by Cooperating Decision Procedures," *ACM TOPLAS* 1(2), 1979 (combination of native decision procedures under explicit compatibility hypotheses rather than a universal solver rank).
- B. Ekici, A. Mebsout, C. Tinelli, C. Keller, G. Katz, A. Reynolds, and C. Barrett, "SMTCoq: A Plug-In for Integrating SMT Solvers into Coq," CAV 2017 (native automation behind a proved certificate-checking boundary).
- G. Morrisett, D. Walker, K. Crary, and N. Glew, "From System F to Typed Assembly Language," *TOPLAS* 21(3), 1999; G. C. Necula and P. Lee, "Safe Kernel Extensions Without Run-Time Checking," OSDI 1996 (type-preserving native realization and validate-once execution).
- M. Stay and L. G. Meredith, "Higher Category Models of the π-Calculus," 2015; B. Toninho and N. Yoshida, "Depending on Session-Typed Processes," FoSSaCS 2018 (typed higher operational cells and value-dependent process protocols).
- J. Hayman and G. Winskel, "Independence and Concurrent Separation Logic," 2008; C. Nester, "The Structure of Concurrent Process Histories," 2021 (proof-relevant independence and equipment-like material histories).
- T. Winterhalter et al., "Eliminating Reflection from Type Theory," POPL 2019 (extensional-to-intensional translation by explicit transports), together with the `ett-to-itt` formalization and Lean4Less's adaptation.
- `Mettapedia/GSLT/LanguageDef/KernelAuthority.lean` and `NIKMetalogic.lean` (direct kernels, exact proof fibres, contextual and resource doctrines, admitted operations, impossibility frontier, bootstrap).
- `Mettapedia/Languages/MeTTa/NativeTypeTheoryDerivation.lean` (dialect observations and commitments, native modal DTT, Pure refinement, equality profiles, Prime returned-fibre bridge, exact parity audit).
- `Mettapedia/Languages/MeTTa/Prime/DataFibration.lean` and `DataTranslationKernel.lean` (non-idempotent language-indexed Data, validated-language structural action, chosen strongly cocartesian push-forwards, four entry modes, syntax-axis separation, exact first dialect delta, and contextual opening/closing/substitution hygiene).
- `ProofGSLT` names the class of proof-supporting GSLTs. The explicit generic checker is the *ProofGSLT reference realization*; these names are not interchangeable.